

Глубокое обучение — это вид машинного обучения, наделяющий компьютеры способностью учиться на опыте. Книга содержит математические и концептуальные основы линейной алгебры, теории вероятностей и теории информации, численных расчетов и машинного обучения в том объеме, который необходим для понимания материала. Описываются приемы глубокого обучения, применяемые на практике, в том числе глубокие сети прямого распространения, регуляризация, алгоритмы оптимизации, сверточные сети, моделирование последовательностей и др. Рассматриваются такие приложения, как обработка естественных языков, распознавание речи, компьютерное зрение, онлайнные рекомендательные системы, биоинформатика и видеоигры. Наконец, описываются перспективные направления исследований: линейные факторные модели, автокодировщики, обучение представлений, структурные вероятностные модели, методы Монте-Карло, статистическая сумма, приближенный вывод и глубокие порождающие модели.

Издание будет полезно студентами и аспирантам, а также опытным программистам, которые хотели бы применить глубокое обучение в составе своих продуктов или платформ.



Ян Гудфеллоу (Ian Goodfellow), исследователь, работающий в компании OpenAI. Изобрел различные алгоритмы машинного обучения, в т.ч. порождающие состязательные сети, и внес вклад в различные программы машинного обучения, включая библиотеки TensorFlow и Theano.



Иошуа Бенджио (Yoshua Bengio), профессор факультета информатики и исследования операций, директор Монреальского института алгоритмов обучения. Основная цель его исследований — понять те принципы обучения, которые порождают интеллект. Читает курс по машинному обучению и руководит большой группой студентов и аспирантов. Редактор журнала Journal of Machine Learning Research, заместитель редактора журнала Neural Computation. Участвовал в организации различных конференций, семинаров и симпозиумов по машинному обучению.



Аарон Курвилль (Aaron Courville), доцент факультета информатики и исследования операций в Монреальском университете, член Монреальского института алгоритмов обучения (MILA).

Интернет-магазин:
www.dmkpress.com
Книга — почтой:
e-mail: orders@aliens-kniga.ru
Оптовая продажа:
«Альянс-книга»
Тел./факс: (499) 782-3889
e-mail: books@aliens-kniga.ru



ISBN 978-5-97060-618-6



Глубокое обучение

«Книга написана ведущими специалистами в этой области и представляет собой единственное полное изложение предмета».

— Илон Маск,
сооснователь компаний Tesla и SpaceX



**Цветное
издание**



Я. Гудфеллоу, И. Бенджио, А. Курвилль

Глубокое обучение

Ян Гудфеллоу, Иошуа Бенджио, Аарон Курвилль

Глубокое обучение

Ian Goodfellow, Yoshua Bengio, Aaron Courville

Deep Learning

The MIT Press
Cambridge, Massachusetts
London, England

Ян Гудфеллоу, Иошуа Бенджио, Аарон Курвилль

Глубокое обучение

Второе цветное издание, исправленное



Москва, 2018

УДК 004.85
ББК 32.971.3
Г93

Гудфеллоу Я., Бенджио И., Курвилль А.
Г93 Глубокое обучение / пер. с англ. А. А. Слинкина. – 2-е изд., испр. – М.: ДМК Пресс, 2018. – 652 с.: цв. ил.

ISBN 978-5-97060-618-6

Глубокое обучение — это вид машинного обучения, наделяющий компьютеры способностью учиться на опыте и понимать мир в терминах иерархии концепций. Книга содержит математические и концептуальные основы линейной алгебры, теории вероятностей и теории информации, численных расчетов и машинного обучения в том объеме, который необходим для понимания материала. Описываются приемы глубокого обучения, применяемые на практике, в том числе глубокие сети прямого распространения, регуляризация, алгоритмы оптимизации, сверточные сети, моделирование последовательностей и др. Рассматриваются такие приложения, как обработка естественных языков, распознавание речи, компьютерное зрение, онлайн-овые рекомендательные системы, биоинформатика и видеоигры.

Издание предназначено студентам вузов и аспирантам, а также опытным программистам, которые хотели бы применить глубокое обучение в составе своих продуктов или платформ.

УДК 004.85
ББК 32.971.3

Права на издание книги на русском языке предоставлены агентством Александра Корженевского.

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

Материал, изложенный в данной книге, многократно проверен. Но поскольку вероятность технических ошибок все равно существует, издательство не может гарантировать абсолютную точность и правильность приводимых сведений. В связи с этим издательство не несет ответственности за возможные ошибки, связанные с использованием книги.

ISBN 978-1-491-93799-0 (анг.)
ISBN 978-5-97060-618-6 (рус.)

© 2017 Massachusetts Institute of Technology
© Оформление, издание, перевод, ДМК Пресс, 2018

Содержание

Веб-сайт	14
Благодарности	15
Обозначения	18
 Глава 1. Введение.....	 21
1.1. На кого ориентирована эта книга	29
1.2. Исторические тенденции в машинном обучении	29
1.2.1. Нейронные сети: разные названия и переменчивая фортуна	30
1.2.2. Увеличение размера набора данных	36
1.2.3. Увеличение размера моделей	36
1.2.4. Повышение точности и сложности и расширение круга задач	40
 Часть I. Основы прикладной математики и машинного обучения	 43
 Глава 2. Линейная алгебра	 44
2.1. Скаляры, векторы, матрицы и тензоры	44
2.2. Умножение матриц и векторов	46
2.3. Единичная и обратная матрица	47
2.4. Линейная зависимость и линейная оболочка	48
2.5. Нормы	50
2.6. Специальные виды матриц и векторов	51
2.7. Спектральное разложение матрицы	52
2.8. Сингулярное разложение	54
2.9. Псевдообратная матрица Мура–Пенроуза	55
2.10. Оператор следа	56
2.11. Определитель	56
2.12. Пример: метод главных компонент	57
 Глава 3. Теория вероятностей и теория информации	 61
3.1. Зачем нужна вероятность?	61
3.2. Случайные величины	63
3.3. Распределения вероятности	63
3.3.1. Дискретные случайные величины и функции вероятности	64
3.3.2. Непрерывные случайные величины и функции плотности вероятности	64
3.4. Маргинальное распределение вероятности	65
3.5. Условная вероятность	65
3.6. Цепное правило	66
3.7. Независимость и условная независимость	66
3.8. Математическое ожидание, дисперсия и ковариация	66
3.9. Часто встречающиеся распределения вероятности	68
3.9.1. Распределение Бернулли	68

3.9.2. Категориальное распределение.....	68
3.9.3. Нормальное распределение.....	69
3.9.4. Экспоненциальное распределение и распределение Лапласа	70
3.9.5. Распределение Дирака и эмпирическое распределение.....	71
3.9.6. Смеси распределений	71
3.10. Полезные свойства употребительных функций	73
3.11. Правило Байеса	74
3.12. Технические детали непрерывных величин	75
3.13. Теория информации.....	76
3.14. Структурные вероятностные модели.....	78
Глава 4. Численные методы	82
4.1. Переполнение и потеря значимости.....	82
4.2. Плохая обусловленность	83
4.3. Оптимизация градиентным методом	84
4.3.1. Не только градиент: матрицы Якоби и Гессе.....	86
4.4. Оптимизация с ограничениями	92
4.5. Пример: линейный метод наименьших квадратов	94
Глава 5. Основы машинного обучения.....	96
5.1. Алгоритмы обучения.....	97
5.1.1. Задача Т	97
5.1.2. Мера качества Р.....	100
5.1.3. Опыт Е	101
5.1.4. Пример: линейная регрессия	103
5.2. Емкость, переобучение и недообучение	105
5.2.1. Теорема об отсутствии бесплатных завтраков.....	110
5.2.2. Регуляризация	112
5.3. Гиперпараметры и контрольные наборы.....	114
5.3.1. Перекрестная проверка.....	115
5.4. Оценки, смещение и дисперсия.....	115
5.4.1. Точечное оценивание	116
5.4.2. Смещение.....	117
5.4.3. Дисперсия и стандартная ошибка.....	119
5.4.4. Поиск компромисса между смещением и дисперсией для минимизации среднеквадратической ошибки.....	121
5.4.5. Состоятельность.....	122
5.5. Оценка максимального правдоподобия	122
5.5.1. Условное логарифмическое правдоподобие и среднеквадратическая ошибка.....	123
5.5.2. Свойства максимального правдоподобия	125
5.6. Байесовская статистика	125
5.6.1. Оценка апостериорного максимума (MAP).....	128
5.7. Алгоритмы обучения с учителем.....	129
5.7.1. Вероятностное обучение с учителем.....	129

5.7.2. Метод опорных векторов.....	130
5.7.3. Другие простые алгоритмы обучения с учителем	132
5.8. Алгоритмы обучения без учителя	134
5.8.1. Метод главных компонент.....	135
5.8.2. Кластеризация методом k средних	137
5.9. Стохастический градиентный спуск	138
5.10. Построение алгоритма машинного обучения	140
5.11. Проблемы, требующие глубокого обучения.....	141
5.11.1. Проклятие размерности	141
5.11.2. Регуляризация для достижения локального постоянства и гладкости.....	142
5.11.3. Обучение многообразий.....	145
Часть II. Глубокие сети: современные подходы.....	149
Глава 6. Глубокие сети прямого распространения	150
6.1. Пример: обучение XOR	152
6.2. Обучение градиентными методами	157
6.2.1. Функции стоимости.....	158
6.2.2. Выходные блоки	160
6.3. Скрытые блоки.....	169
6.3.1. Блоки линейной ректификации и их обобщения.....	170
6.3.2. Логистическая сигмоида и гиперболический тангенс	171
6.3.3. Другие скрытые блоки	172
6.4. Проектирование архитектуры	173
6.4.1. Свойства универсальной аппроксимации и глубина	174
6.4.2. Другие архитектурные подходы	177
6.5. Обратное распространение и другие алгоритмы дифференцирования.....	179
6.5.1. Графы вычислений	179
6.5.2. Правило дифференцирования сложной функции	181
6.5.3. Рекурсивное применение правила дифференцирования сложной функции для получения алгоритма обратного распространения.....	182
6.5.4. Вычисление обратного распространения в полносвязном МСП	185
6.5.5. Символьно-символьные производные	186
6.5.6. Общий алгоритм обратного распространения.....	188
6.5.7. Пример: применение обратного распространения к обучению МСП.....	191
6.5.8. Осложнения	192
6.5.9. Дифференцирование за пределами сообщества глубокого обучения	193
6.5.10. Производные высшего порядка	195
6.6. Исторические замечания	196
Глава 7. Регуляризация в глубоком обучении	199
7.1. Штрафы по норме параметров	200
7.1.1. Регуляризация параметров по норме L^2	201
7.1.2. L^1 -регуляризация.....	204
7.2. Штраф по норме как оптимизация с ограничениями	206

7.3. Регуляризация и недоопределенные задачи.....	208
7.4. Пополнение набора данных.....	208
7.5. Робастность относительно шума.....	210
7.5.1. Привнесение шума в выходные метки.....	211
7.6. Обучение с частичным привлечением учителя.....	211
7.7. Многозадачное обучение.....	212
7.8. Ранняя остановка.....	213
7.9. Связывание и разделение параметров.....	219
7.9.1. Сверточные нейронные сети.....	220
7.10. Разреженные представления.....	220
7.11. Баггинг и другие ансамблевые методы.....	222
7.12. Прореживание.....	224
7.13. Состязательное обучение.....	232
7.14. Тангенциальное расстояние, алгоритм распространения по касательной и классификатор по касательной к многообразию.....	233
Глава 8. Оптимизация в обучении глубоких моделей.....	237
8.1. Чем обучение отличается от чистой оптимизации.....	237
8.1.1. Минимизация эмпирического риска.....	238
8.1.2. Суррогатные функции потерь и ранняя остановка.....	239
8.1.3. Пакетные и мини-пакетные алгоритмы.....	239
8.2. Проблемы оптимизации нейронных сетей.....	243
8.2.1. Плохая обусловленность.....	243
8.2.2. Локальные минимумы.....	245
8.2.3. Плато, седловые точки и другие плоские участки.....	246
8.2.4. Утесы и резко растущие градиенты.....	248
8.2.5. Долгосрочные зависимости.....	249
8.2.6. Неточные градиенты.....	250
8.2.7. Плохое соответствие между локальной и глобальной структурами.....	250
8.2.8. Теоретические пределы оптимизации.....	252
8.3. Основные алгоритмы.....	253
8.3.1. Стохастический градиентный спуск.....	253
8.3.2. Импульсный метод.....	255
8.3.3. Метод Нестерова.....	258
8.4. Стратегии инициализации параметров.....	258
8.5. Алгоритмы с адаптивной скоростью обучения.....	263
8.5.1. AdaGrad.....	264
8.5.2. RMSProp.....	264
8.5.3. Adam.....	265
8.5.4. Выбор правильного алгоритма оптимизации.....	266
8.6. Приближенные методы второго порядка.....	267
8.6.1. Метод Ньютона.....	267
8.6.2. Метод сопряженных градиентов.....	268
8.6.3. Алгоритм BFGS.....	271
8.7. Стратегии оптимизации и метаалгоритмы.....	272

8.7.1. Пакетная нормировка.....	272
8.7.2. Покоординатный спуск.....	275
8.7.3. Усреднение Поляка.....	276
8.7.4. Предобучение с учителем.....	276
8.7.5. Проектирование моделей с учетом простоты оптимизации.....	279
8.7.6. Методы продолжения и обучение по плану.....	279
Глава 9. Сверточные сети.....	282
9.1. Операция свертки.....	282
9.2. Мотивация.....	284
9.3. Пулинг.....	290
9.4. Свертка и пулинг как бесконечно сильное априорное распределение.....	293
9.5. Варианты базовой функции свертки.....	295
9.6. Структурированный выход.....	304
9.7. Типы данных.....	305
9.8. Эффективные алгоритмы свертки.....	306
9.9. Случайные признаки и признаки, обученные без учителя.....	307
9.10. Нейробиологические основания сверточных сетей.....	308
9.11. Сверточные сети и история глубокого обучения.....	314
Глава 10. Моделирование последовательностей: рекуррентные и рекурсивные сети.....	316
10.1. Развертка графа вычислений.....	317
10.2. Рекуррентные нейронные сети.....	320
10.2.1. Форсирование учителя и сети с рекурсией на выходе.....	323
10.2.2. Вычисление градиента в рекуррентной нейронной сети.....	325
10.2.3. Рекуррентные сети как ориентированные графические модели.....	327
10.2.4. Моделирование контекстно-обусловленных последовательностей с помощью РНС.....	330
10.3. Двухнаправленные РНС.....	332
10.4. Архитектуры кодировщик-декодер или последовательность в последовательность.....	333
10.5. Глубокие рекуррентные сети.....	336
10.6. Рекурсивные нейронные сети.....	337
10.7. Проблема долгосрочных зависимостей.....	339
10.8. Нейронные эхо-сети.....	341
10.9. Блоки с утечками и другие стратегии нескольких временных масштабов.....	343
10.9.1. Добавление прямых связей сквозь время.....	343
10.9.2. Блоки с утечкой и спектр разных временных масштабов.....	343
10.9.3. Удаление связей.....	344
10.10. Долгая краткосрочная память и другие вентильные РНС.....	344
10.10.1. Долгая краткосрочная память.....	345
10.10.2. Другие вентильные РНС.....	347
10.11. Оптимизация в контексте долгосрочных зависимостей.....	348
10.11.1. Отсечение градиентов.....	348

10.11.2. Регуляризация с целью подталкивания информационного потока	350
10.12. Явная память	351
Глава 11. Практическая методология	355
11.1. Показатели качества	356
11.2. Выбор базовой модели по умолчанию	358
11.3. Надо ли собирать дополнительные данные?	359
11.4. Выбор гиперпараметров	360
11.4.1. Ручная настройка гиперпараметров	360
11.4.2. Алгоритмы автоматической оптимизации гиперпараметров	363
11.4.3. Поиск на сетке	364
11.4.4. Случайный поиск	365
11.4.5. Оптимизация гиперпараметров на основе модели	366
11.5. Стратегии отладки	367
11.6. Пример: распознавание нескольких цифр	370
Глава 12. Приложения	373
12.1. Крупномасштабное глубокое обучение	373
12.1.1. Реализации на быстрых CPU	373
12.1.2. Реализации на GPU	374
12.1.3. Крупномасштабные распределенные реализации	376
12.1.4. Сжатие модели	376
12.1.5. Динамическая структура	377
12.1.6. Специализированные аппаратные реализации глубоких сетей	379
12.2. Компьютерное зрение	380
12.2.1. Предобработка	381
12.3. Распознавание речи	385
12.4. Обработка естественных языков	388
12.4.1. n -граммы	388
12.4.2. Нейронные языковые модели	390
12.4.3. Многомерные выходы	391
12.4.4. Комбинирование нейронных языковых моделей с n -граммами	397
12.4.5. Нейронный машинный перевод	397
12.4.6. Историческая справка	401
12.5. Другие приложения	402
12.5.1. Рекомендательные системы	402
12.5.2. Представление знаний, рассуждения и ответы на вопросы	405
Часть III. Исследования по глубокому обучению	409
Глава 13. Линейные факторные модели	411
13.1. Вероятностный РСА и факторный анализ	412
13.2. Анализ независимых компонент (ICA)	413
13.3. Анализ медленных признаков	415
13.4. Разреженное кодирование	417
13.5. Интерпретация РСА в терминах многообразий	419

Глава 14. Автокодировщики	422
14.1. Понижающие автокодировщики	423
14.2. Регуляризованные автокодировщики	423
14.2.1. Разреженные автокодировщики	424
14.2.2. Шумоподавляющие автокодировщики	426
14.2.3. Регуляризация посредством штрафования производных	427
14.3. Репрезентативная способность, размер слоя и глубина	427
14.4. Стохастические кодировщики и декодеры	428
14.5. Шумоподавляющие автокодировщики	429
14.5.1. Сопоставление рейтингов	430
14.6. Обучение многообразий с помощью автокодировщиков	433
14.7. Сжимающие автокодировщики	436
14.8. Предсказательная разреженная декомпозиция	440
14.9. Применения автокодировщиков	441
Глава 15. Обучение представлений	443
15.1. Жадное послойное предобучение без учителя	444
15.1.1. Когда и почему работает предобучение без учителя?	446
15.2. Перенос обучения и адаптация домена	451
15.3. Разделение каузальных факторов с частичным привлечением учителя	454
15.4. Распределенное представление	459
15.5. Экспоненциальный выигрыш от глубины	465
15.6. Ключ к выявлению истинных причин	466
Глава 16. Структурные вероятностные модели в глубоком обучении	469
16.1. Проблема бесструктурного моделирования	470
16.2. Применение графов для описания структуры модели	473
16.2.1. Ориентированные модели	473
16.2.2. Неориентированные модели	475
16.2.3. Статистическая сумма	477
16.2.4. Энергетические модели	478
16.2.5. Разделенность и d-разделенность	480
16.2.6. Преобразование между ориентированными и неориентированными графами	481
16.2.7. Факторные графы	486
16.3. Выборка из графических моделей	487
16.4. Преимущества структурного моделирования	488
16.5. Обучение и зависимости	489
16.6. Вывод и приближенный вывод	490
16.7. Подход глубокого обучения к структурным вероятностным моделям	491
16.7.1. Пример: ограниченная машина Больцмана	492
Глава 17. Методы Монте-Карло	495
17.1. Выборка и методы Монте-Карло	495
17.1.1. Зачем нужна выборка?	495

17.1.2. Основы выборки методом Монте-Карло	495
17.2. Выборка по значимости	497
17.3. Методы Монте-Карло по схеме марковской цепи	499
17.4. Выборка по Гиббсу.....	502
17.5. Проблема перемешивания разделенных мод	503
17.5.1. Применение темперирования для перемешивания мод	506
17.5.2. Глубина может помочь перемешиванию	506

Глава 18. Преодоление трудностей, связанных

со статической суммой.....	508
18.1. Градиент логарифмического правдоподобия	508
18.2. Стохастическая максимизация правдоподобия и сопоставительное расхождение	510
18.3. Псевдоправдоподобие	517
18.4. Сопоставление рейтингов и сопоставление отношений	519
18.5. Шумоподавляющее сопоставление рейтингов.....	521
18.6. Шумосопоставительное оценивание	521
18.7. Оценивание статистической суммы.....	524
18.7.1. Выборка по значимости с отжигом	525
18.7.2. Мостиковая выборка	528

Глава 19. Приближенный вывод.....

19.1. Вывод как оптимизация.....	530
19.2. ЕМ-алгоритм	532
19.3. МАР-вывод и разреженное кодирование	533
19.4. Вариационный вывод и обучение	535
19.4.1. Дискретные латентные переменные	536
19.4.2. Вариационное исчисление.....	541
19.4.3. Непрерывные латентные переменные.....	544
19.4.4. Взаимодействия между обучением и выводом	545
19.5. Обученный приближенный вывод	546
19.5.1. Бодрствование-сон.....	546
19.5.2. Другие формы обученного вывода	547

Глава 20. Глубокие порождающие модели.....

20.1. Машины Больцмана.....	548
20.2. Ограниченные машины Больцмана.....	550
20.2.1. Условные распределения	550
20.2.2. Обучение ограниченных машин Больцмана.....	552
20.3. Глубокие сети доверия	553
20.4. Глубокие машины Больцмана	555
20.4.1. Интересные свойства.....	557
20.4.2. Вывод среднего поля в ГМБ	558
20.4.3. Обучение параметров ГМБ.....	560
20.4.4. Послойное предобучение	560

20.4.5. Совместное обучение глубоких машин Больцмана.....	563
20.5. Машины Больцмана для вещественных данных	566
20.5.1. ОМБ Гаусса–Бернулли	567
20.5.2. Неориентированные модели условной ковариации.....	568
20.6. Сверточные машины Больцмана.....	572
20.7. Машины Больцмана для структурных и последовательных выходов.....	573
20.8. Другие машины Больцмана.....	574
20.9. Обратное распространение через случайные операции	575
20.9.1. Обратное распространение через дискретные стохастические операции.....	577
20.10. Ориентированные порождающие сети	579
20.10.1. Сигмоидные сети доверия.....	580
20.10.2. Дифференцируемые генераторные сети	581
20.10.3. Вариационные автокодировщики	583
20.10.4. Порождающие состязательные сети	586
20.10.5. Порождающие сети с сопоставлением моментов.....	589
20.10.6. Сверточные порождающие сети.....	590
20.10.7. Авторегрессивные сети.....	591
20.10.8. Линейные авторегрессивные сети.....	591
20.10.9. Нейронные авторегрессивные сети.....	592
20.10.10. NADE.....	593
20.11. Выборка из автокодировщиков	595
20.11.1. Марковская цепь, ассоциированная с произвольным шумоподавляющим автокодировщиком	596
20.11.2. Фиксация и условная выборка.....	596
20.11.3. Возвратная процедура обучения.....	597
20.12. Порождающие стохастические сети.....	598
20.12.1. Дискриминантные GSN	599
20.13. Другие схемы порождения	599
20.14. Оценивание порождающих моделей	600
20.15. Заключение	603
Список литературы	604
Предметный указатель	646

Веб-сайт

www.deeplearning.book

Книгу сопровождает указанный выше сайт, где представлены упражнения, слайды, исправления ошибок и другие материалы, полезные читателям и преподавателям.

Благодарности

Эта книга не состоялась бы, если бы не помощь со стороны многих людей.

Мы благодарны тем, кто откликнулся на наше предложение о написании книги и помог спланировать ее содержание и структуру: Гийом Ален (Guillaume Alain), Кюнхюн Чо (Kyunghyun Cho), Чаглар Гюльчехре (Çaglar Gülçehre), Дэвид Крюгер (David Krueger), Гуго Ларошель (Hugo Larochelle), Разван Паскану (Razvan Pascanu) и Томас Рохе (Thomas Rohée).

Мы также благодарны всем, кто присылал отзывы о самой книге, иногда даже о нескольких главах: Мартен Абади (Martín Abadi), Гийом Ален, Йон Андруцопулос (Ion Androutsopoulos), Фред Берш (Fred Bertsch), Олекса Биланюк (Olexa Bilaniuk), Уфук Джан Бичиджи (Ufuk Can Biçici), Матко Босняк (Matko Bošnjak), Джон Буерсма (John Boersma), Грег Брокман (Greg Brockman), Александр де Бребиссон (Alexandre de Brébisson), Пьер Люк Каррье (Pierre Luc Carrier), Саратх Чандар (Sarath Chandar), Павел Чилински (Pawel Chilinski), Марк Дауст (Mark Daoust), Олег Дашевский, Лоран Дин (Laurent Dinh), Стефан Дрезейтль (Stephan Dreseidl), Джим Фан (Jim Fan), Мяо Фан (Miao Fan), Мейре Фортунато (Meire Fortunato), Фредерик Франсис (Frédéric Francis), Нандо де Фрейтас (Nando de Freitas), Чаглар Гюльчехре (Çaglar Gülçehre), Юрген Ван Гаэл (Jurgen Van Gael), Хавьер Алонсо Гарсиа (Javier Alonso García), Джонатан Хант (Jonathan Hunt), Гопи Джейярам (Gopi Jeyaram), Чингиз Кабытаев (Chingiz Kabytayev), Лукаш Кайзер (Lukasz Kaiser), Варун Канаде (Varun Kanade), Азифулла Хан (Asifullah Khan), Акиель Хан (Akiel Khan), Джон Кинг (John King), Дидерик П. Кингма (Diederik P. Kingma), Ян Лекун (Yann LeCun), Рудольф Матей (Rudolf Mathey), Матиас Маттамала (Matías Mattamala), Абхинав Мауриа (Abhinav Maurya), Кэвин Мерфи (Kevin Murphy), Олег Мюрк (Oleg Mürk), Роман Новак (Roman Novak), Огастес К. Одена (Augustus Q. Odena), Симон Павлик (Simon Pavlik), Карл Пичотта (Karl Pichotta), Эдди Пирс (Eddie Pierce), Кари Пулли (Kari Pulli), Руссель Рахман (Roussel Rahman), Тапани Райко (Tapani Raiko), Анурга Ранджан (Anurag Ranjan), Йоханнес Ройт (Johannes Roith), Михаэла Роска (Mihaela Rosca), Халис Сак (Halis Sak), Сезар Салгадо (César Salgado), Григорий Сапунов, Ёсинори Сасаки (Yoshinori Sasaki), Майк Шустер (Mike Schuster), Джулиан Сербан (Julian Serban), Нир Шабат (Nir Shabat), Кен Ширрифф (Ken Shirriff), Андрэ Симпело (Andre Simpelo), Дэвид Слейт (David Slate), Скотт Стэнли (Scott Stanley), Давид Суссилло (David Sussillo), Илья Суцкевер (Ilya Sutskever), Карлес Гелада Саец (Carles Gelada Sáez), Грэхэм Тейлор (Graham Taylor), Валентин Толмер (Valentin Tolmer), Массимильяно Томассоли (Massimiliano Tomassoli), Ан Тран (An Tran), Шубхенду Триведи (Shubhendu Trivedi), Алексей Умнов, Винсет Ванхоуке (Vincent Vanhoucke), Марко Висентини-Скарцанелла (Marco Visentini-Scarzarella), Матрин Вита (Martin Vita), Дэвид Уорд-Фарли (David Warde-Farley), Дастин Уэбб (Dustin Webb), Кэлвин Су (Kelvin Xu), Вэй Сюэ (Wei Xue), Ке Янг (Ke Yang), Ли Яо (Li Yao), Зигмунт Заяц (Zygmunt Zając) и Озан Чаглаян (Ozan Çaglayan).

Мы признательны и тем, кто делился своим мнением об отдельных главах.

- Обозначения: Чжан Юань Хан (Zhang Yuanhang).
- Глава 1 «Введение»: Юсуф Акгуль (Yusuf Akgul), Себастьян Братьерес (Sebastien Bratieres), Самира Эбрахими (Samira Ebrahimi), Чарли Горичаназ (Charlie

Gorichanaz), Брендан Лоудермилк (Brendan Loudermilk), Эрис Моррис (Eric Morris), Космин Пярвулеску (Cosmin Pârvulescu) и Альфредо Солано (Alfredo Solano).

- Глава 2 «Линейная алгебра»: Амджад Алмахаири (Amjad Almahairi), Никола Баниц (Nikola Banic), Кэвин Беннетт (Kevin Bennett), Филипп Кастонге (Philippe Castonguay), Оскар Чанг (Oscar Chang), Эрик Фослер-Люссье (Eric Fosler-Lussier), Андрей Халявин, Сергей Орешков (Sergey Oreshkov), Иштван Петрас (István Petrás), Дэннис Прэнгл (Dennis Prangle), Томас Рохе (Thomas Rohée), Гитанджали Гулве Сехгал (Gitanjali Gulve Sehgal), Колби Толанд (Colby Toland), Алессандро Витале (Alessandro Vitale) и Боб Уэлланд (Bob Welland).
- Глава 3 «Теория вероятностей и теория информации»: Джон Филипп Андерсон (John Philip Anderson), Кай Арулкумаран (Kai Arulkumaran), Венсан Дюмуле (Vincent Dumoulin), Руй Фа (Rui Fa), Стефан Гоус (Stephan Gouws), Артем Оботуров, Антти Расмус (Antti Rasmus), Алексей Сурков и Фолькер Тресп (Volker Tresp).
- Глава 4 «Численные методы»: Тран Лам Аниан Фишер (Tran Lam AnIan Fischer) и Ху Ю Хуан (Hu Yuhuang).
- Глава 5 «Основы машинного обучения»: Дзмитри Бахданау (Dzmitry Bahdanau), Жюстен Доманге (Justin Domingue), Нихил Гарг (Nikhil Garg), Макото Оцука (Makoto Otsuka), Боб Пепин (Bob Pepin), Филип Попьен (Philip Popien), Бхарат Прабхакар (Bharat Prabhakar), Эммануэль Райнер (Emmanuel Rayner), Питер Шепард (Peter Shepard), Ки-Бонг Сонг (Kee-Bong Song), Чжен Сун (Zheng Sun) и Энди Ву (Andy Wu).
- Глава 6 «Глубокие сети прямого распространения»: Уриэль Бердуго (Uriel Berdugo), Фабрицио Боттарель (Fabrizio Bottarel), Элизабет Бэрл (Elizabeth Burl), Ишан Дуругкар (Ishan Durugkar), Джефф Хлыва (Jeff Hlywa), Ёнг Вук Ким (Jong Wook Kim), Давид Крюгер (David Krueger), Адития Кумар Прахарадж (Aditya Kumar Praharaj) и Стэн Сутла (Sten Sootla).
- Глава 7 «Регуляризация в глубоком обучении»: Мортен Колбэк (Morten Kolbæk), Читыдж Лауриа (Kshitij Lauria), Инкю Ли (Inkyu Lee), Сунил Мохан (Sunil Mohan), Хай Пхонг Пхан (Hai Phong Phan) и Джошуа Сэлисбэри (Joshua Salisbury).
- Глава 8 «Оптимизация обучения глубоких моделей»: Марсель Аккерман (Marcel Ackermann), Питер Армитейдж (Peter Armitage), Роуэл Атиенза (Rowel Atienza), Эндрю Брок (Andrew Brock), Теган Махарадж (Tegan Maharaj), Джеймс Мартенс (James Martens), Мостафа Натех (Mostafa Nategh), Кашиф Расул (Kashif Rasul), Клаус Штробль (Klaus Strobl) и Никола Тэрнер (Nicholas Turner).
- Глава 9 «Сверточные сети»: Мартен Аржовски (Martín Arjovsky), Евгений Бревдо (Eugene Brevdo), Константин Дивилов, Эрик Йенсен (Eric Jensen), Мехди Мирза (Mehdi Mirza), Алекс Пайно (Alex Paino), Марджори Сэйер (Marjorie Sayer), Райан Стаут (Ryan Stout) и Вентао Ву (Wentao Wu).
- Глава 10 «Моделирование последовательностей: рекуррентные и рекурсивные сети»: Гёкчен Ераслан (Gökçen Eraslan), Стивен Хиксон (Steven Hickson), Разван Паскану (Razvan Pascanu), Лоренхо фон Риттер (Lorenzo von Ritter), Руй Родригес (Rui Rodrigues), Дмитрий Сердюк, Донгуй Ши (Dongyu Shi) и Кай Ю Ян (Kaiyu Yang).

- Глава 11 «Практическая методология»: Даниэль Бекштейн (Daniel Beckstein).
- Глава 12 «Приложения»: Джордж Дал (George Dahl), Владимир Некрасов (Vladimir Nekrasov) и Рибана Рошер (Ribana Roscher).
- Глава 13 «Линейные факторные модели»: Джейнт Кушик (Jayanth Koushik).
- Глава 15 «Обучение представлений»: Кунал Гхош (Kunal Ghosh).
- Глава 16 «Структурные вероятностные модели в глубоком обучении»: Минь Ле (Minh Lê) и Антон Варфолом (Anton Varfolom).
- Глава 18 «Преодоление трудностей, связанных со статической суммой»: Сэм Боумен (Sam Bowman).
- Глава 19 «Приближенный вывод»: Ю Цзя Бао (Yujia Bao).
- Глава 20 «Глубокие порождающие модели»: Николас Чападос (Nicolas Charados), Даниэль Галвес (Daniel Galvez), Вэн Мин Ма (Wenming Ma), Фади Медхат (Fady Medhat), Шакри Мохамед (Shakir Mohamed) и Грегуар Монтавон (Grégoire Montavon).
- Библиография: Лукас Михельбахер (Lukas Michelbacher) и Лесли Н. Смит (Leslie N. Smith).

Мы также благодарим авторов, давших нам разрешение использовать в тексте изображения, рисунки и данные из их публикаций. Источники указываются в подписанных подписях.

Мы благодарны Лю Вану (Lu Wang), написавшему программу pdf2htmlEX, которой мы пользовались для подготовки варианта книги для веба, а также за помощь в улучшении качества получившегося HTML-кода.

Спасибо супруге Яна Даниэле Флори Гудфеллоу за терпеливую поддержку Яна на всем протяжении работы над книгой и за помощь в выверке текста.

Спасибо команде Google Brain за создание атмосферы интеллектуального общения, позволившей Яну уделять много времени работе над книгой и получать отзывы и наставления от коллег. Особенно хотим поблагодарить бывшего начальника Яна, Грегга Коррадо (Greg Corrado), и его нынешнего начальника, Сами Бенджио (Samy Bengio), за поддержку этого проекта. Наконец, мы благодарны Джеффри Хинтону (Geoffrey Hinton), который подбадривал нас, когда было трудно.

Обозначения

В этом разделе приведен краткий перечень обозначений, используемых в книге. Большая часть соответствующего математического аппарата описана в главах 2–4.

Числа и массивы

a	скаляр (целый или вещественный)
$\mathbf{a}$	вектор
$\mathbf{A}$	матрица
$\mathbf{A}$	тензор
$\mathbf{I}_n$	единичная матрица с n строками и n столбцами
$\mathbf{I}$	единичная матрица, размер которой определяется контекстом
$\mathbf{e}^{(i)}$	стандартный базисный вектор $[0, \dots, 0, 1, 0, \dots, 0]$, содержащий 1 в i -й позиции
$\text{diag}(\mathbf{a})$	квадратная диагональная матрица, на диагонали которой находятся элементы вектора $\mathbf{a}$
a	случайная скалярная величина
$\mathbf{a}$	случайный вектор
$\mathbf{A}$	случайная матрица

Множества и графы

$\mathbb{A}$	множество
$\mathbb{R}$	множество вещественных чисел
$\{0, 1\}$	множество из двух элементов: 0 и 1
$\{0, 1, \dots, n\}$	множество целых чисел от 0 до n включительно
$[a, b]$	замкнутый интервал вещественной прямой от a до b , включающий границы
$(a, b]$	интервал вещественной прямой, не включающий a , но включающий b
$\mathbb{A} \setminus \mathbb{B}$	разность множеств, т. е. множество, содержащее все элементы $\mathbb{A}$, не являющиеся элементами $\mathbb{B}$
$\mathcal{G}$	граф
$\text{Pa}_{\mathcal{G}}(x_i)$	родители x_i в $\mathcal{G}$

Индексирование

a_i	i -й элемент вектора $\mathbf{a}$, индексирование начинается с 1
a_{-i}	все элементы вектора $\mathbf{a}$, кроме i -го
$A_{i,j}$	элемент матрицы $\mathbf{A}$ в позиции (i, j)
$A_{i,:}$	i -я строка матрицы $\mathbf{A}$
$A_{:,i}$	i -й столбец матрицы $\mathbf{A}$
$A_{i,j,k}$	элемент трехмерного тензора $\mathbf{A}$ в позиции (i, j, k)
$\mathbf{A}_{:, :, i}$	двумерная срезка трехмерного тензора $\mathbf{A}$
$\mathbf{a}_i$	i -й элемент случайного вектора $\mathbf{a}$

Операции линейной алгебры

$\mathbf{A}^{\top}$	матрица, транспонированная к $\mathbf{A}$
$\mathbf{A}^{+}$	псевдообратная матрица Мура-Пенроуза

$A \odot B$	поэлементное произведение матриц A и B (произведение Адамара)
$\det(A)$	определитель A

Математический анализ

$\frac{dy}{dx}$	производная y по x
$\frac{\partial y}{\partial x}$	частная производная y по x
$\nabla_x y$	градиент y по x
$\nabla_x y$	матрица производных y относительно X
$\nabla_x y$	тензор производных y относительно X
$\frac{\partial f}{\partial x}$	матрица Якоби $J \in \mathbb{R}^{m \times n}$ функции $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$
$\nabla_x^2 f(x)$ или $H(f)(x)$	гессиан функции f в точке x
$\int f(x) dx$	определенный интеграл по всей области определения x
$\int_S f(x) dx$	определенный интеграл по множеству S

Теория вероятностей и теория информации

$a \perp b$	случайные величины a и b независимы
$a \perp b \mid c$	они условно независимы при условии c
$P(a)$	распределение вероятности дискретной случайной величины
$p(a)$	распределение вероятности непрерывной случайной величины или величины, тип которой не задан
$a \sim P$	случайная величина a имеет распределение P
$\mathbb{E}_{x \sim P}[f(x)]$ или $\mathbb{E}f(x)$	математическое ожидание $f(x)$ при заданном распределении $P(x)$
$\text{Var}(f(x))$	дисперсия $f(x)$ при заданном распределении $P(x)$
$\text{Cov}(f(x), g(x))$	ковариация $f(x)$ и $g(x)$ при заданном распределении $P(x)$
$H(x)$	энтропия Шеннона случайной величины x
$D_{\text{KL}}(P \parallel Q)$	расхождение Кульбака–Лейблера между P и Q
$N(x; \mu, \Sigma)$	нормальное распределение случайной величины x со средним μ и ковариацией Σ

Функции

$f: A \rightarrow B$	функция f с областью определения A и областью значений B
$f \circ g$	композиция функций f и g
$f(x; \theta)$	функция от x , параметризованная θ (иногда, чтобы не утяжелять формулы, мы пишем $f(x)$, опуская аргумент θ)
$\log x$	натуральный логарифм x
$\sigma(x)$	логистическая сигмоида, $1 / (1 + \exp(-x))$
$\xi(x)$	функция $\log(1 + \exp(x))$
$\ x\ _p$	норма L^p вектора x
$\ x\ $	норма L^2 вектора x
x^+	положительная часть x , т. е. $\max(0, x)$
$1_{\text{condition}}$	1, если условие <i>condition</i> истинно, иначе 0

Иногда мы применяем функцию f со скалярным аргументом к вектору, матрице или тензору: $f(\mathbf{x})$, $f(\mathbf{X})$ или $f(\mathbf{X})$. Это означает, что функция f применяется к каждому элементу массива. Например, запись $\mathbf{C} = \sigma(\mathbf{X})$ означает, что $C_{i,j,k} = \sigma(X_{i,j,k})$ для всех i, j, k .

Наборы данных и распределения

p_{data}	распределение, порождающее данные
$\hat{p}_{\text{data}}$	эмпирическое распределение, определенное обучающим набором
$\mathcal{X}$	обучающий набор примеров
$\mathbf{x}^{(i)}$	i -й пример из входного набора данных
$y^{(i)}$ или $\mathbf{y}^{(i)}$	метка, ассоциированная с $\mathbf{x}^{(i)}$ при обучении с учителем
$\mathbf{X}$	матрица $m \times n$, в строке $\mathbf{X}_{i,:}$ которой находится входной пример $\mathbf{x}^{(i)}$

Изобретатели давно мечтали создать думающую машину. Эти мечты восходят еще к Древней Греции. Персонажей мифов – Пигмалиона, Дедала, Гефеста – можно было бы назвать легендарными изобретателями, а их творения – Галатею, Талоса и Пандору – искусственной жизнью (Ovid and Martin, 2004; Sparkes, 1996; Tandy, 1997).

Впервые задумавшись о программируемых вычислительных машинах, человек задался вопросом, смогут ли они стать разумными, – за сотню с лишним лет до построения компьютера (Lovelace, 1842). Сегодня **искусственный интеллект (ИИ)** – бурно развивающаяся дисциплина, имеющая многочисленные приложения. Мы хотим иметь интеллектуальные программы, которые могли бы автоматизировать рутинный труд, понимали речь и изображения, ставили медицинские диагнозы и поддерживали научные исследования.

Когда наука об искусственном интеллекте только зарождалась, были быстро исследованы и решены некоторые задачи, трудные для человека, но относительно простые для компьютеров – описываемые с помощью списка формальных математических правил. Настоящим испытанием для искусственного интеллекта стали задачи, которые легко решаются человеком, но с трудом поддаются формализации, – задачи, которые мы решаем интуитивно, как бы автоматически: распознавание устной речи или лиц на картинке.

Эта книга посвящена решению таких интуитивных задач. Цель заключается в том, чтобы компьютер мог учиться на опыте и понимать мир в терминах иерархии понятий, каждое из которых определено через более простые понятия. Благодаря приобретению знаний опытным путем этот подход позволяет исключить этап формального описания человеком всех необходимых компьютеру знаний. Иерархическая организация дает компьютеру возможность учиться более сложным понятиям путем построения их из более простых. Граф, описывающий эту иерархию, будет глубоким – содержащим много уровней. Поэтому такой подход к ИИ называется **глубоким обучением**.

Ранние успехи ИИ в большинстве своем были достигнуты в относительно стерильной формальной среде, где от компьютера не требовались обширные знания о мире. Взять, к примеру, созданную IBM шахматную программу Deep Blue, которая в 1997 году обыграла чемпиона мира Гарри Каспарова (Hsu, 2002). Шахматы – это очень простой мир, состоящий всего из 64 клеток и 32 фигур, которые могут ходить лишь строго определенным образом. Разработка успешной стратегии игры в шахматы – огромное достижение, но трудность задачи – не в описании множества фигур и допустимых ходов на языке, понятном компьютеру. Для полного описания игры достаточно очень короткого списка формальных правил, который заранее составляется программистом.

Забавно, что абстрактные, формально поставленные задачи, требующие значительных умственных усилий от человека, для компьютера как раз наиболее просты. Компьютеры давно уже способны обыграть в шахматы сильнейших гроссмейстеров, но лишь в последние годы стали сопоставимы с человеком в части распознавания объектов или речи. В повседневной жизни человеку необходим гигантский объем знаний о мире. Знания эти субъективны и представлены на интуитивном уровне, поэтому выразить их формально затруднительно. Но чтобы вести себя «разумно», компьютерам нужны такие же знания. Одна из основных проблем искусственного интеллекта – как заложить эти неформальные знания в компьютер.

Авторы нескольких проектов в области ИИ пытались представить знания о мире с помощью формальных языков. Компьютер может автоматически рассуждать о предложениях на таком языке, применяя правила логического вывода. В основе таких подходов лежит **база знаний**. Ни один из этих проектов не привел к существенному успеху. Одним из самых известных был проект Сус (Lenat and Guha, 1989) – машина логического вывода и база утверждений на языке СусL. За ввод утверждений отвечал штат учителей-людей. Процесс оказывается крайне громоздким. Люди из всех сил пытаются придумать формальные правила, достаточно сложные для точного описания мира. Например, Сус не сумел понять рассказ о человеке по имени Фред, который бреется по утрам (Linde, 1992). Его машина вывода обнаружила в рассказе противоречие: он знал, что в людях нет электрических деталей, но, поскольку Фред держал электрическую бритву, система решила, что объект «Бреющийся Фред» содержит электрические детали. И задала вопрос: является ли Фред по-прежнему человеком, когда бреется.

Трудности, с которыми сталкиваются системы, опирающиеся на «зашифрованные в код» знания, наводят на мысль, что система с искусственным интеллектом должна уметь самостоятельно накапливать знания, отыскивая закономерности в исходных данных. Это умение называется **машинным обучением**. С появлением машинного обучения перед компьютерами открылась возможность подступиться к задачам, требующим знаний о реальном мире, и принимать решения, кажущиеся субъективными. Простой алгоритм машинного обучения – **логистическая регрессия** – может решить, следует ли рекомендовать кесарево сечение (Mor-Yosef et al., 1990). Другой простой алгоритм – **наивный байесовский классификатор** – умеет отделять нормальную электронную почту от спама.

Качество этих простых алгоритмов сильно зависит от представления исходных данных. Так, система ИИ, выдающая рекомендации о показанности кесарева сечения, не осматривает пациента. Вместо этого врач сообщает системе относящуюся к делу информацию, например о наличии или отсутствии рубца на матке. Каждый отдельный элемент информации, включаемый в представление о пациенте, называется **признаком**. Алгоритм логистической регрессии анализирует, как признаки пациента коррелируют с различными результатами. Но он не может никаким образом повлиять на определение признаков. Если алгоритму предложить снимок МРТ, а не формализованные врачом сведения, то он не сможет выдать полезную рекомендацию. Отдельные пиксели снимка практически не коррелированы с осложнениями, которые могут возникнуть во время родов.

Эта зависимость от представления является общим явлением, проявляющимся как в информатике, так и в повседневной жизни. Если говорить об информатике, то такие операции, как поиск в коллекции данных, будут производиться многократно

быстрее, если коллекция структурирована и подходящим образом индексирована. Люди же легко выполняют арифметические операции с числами, записанными арабскими цифрами, но тратят куда больше времени, если используются римские цифры. Неудивительно, что выбор представления оказывает огромное влияние на качество и производительность алгоритмов машинного обучения. На рис. 1.1 приведен простой наглядный пример.

Многие задачи ИИ можно решить, если правильно подобрать признаки, а затем предъявить их алгоритму машинного обучения. Например, в задаче идентификации говорящего по звукам речи полезным признаком является речевой тракт. Он позволяет с большой точностью определить, кто говорит: мужчина, женщина или ребенок.

Но во многих задачах нелегко понять, какие признаки выделять. Допустим, к примеру, что мы пишем программу обнаружения автомобилей на фотографиях. Мы знаем, что у автомобилей есть колеса, поэтому могли бы считать присутствие колеса признаком. К сожалению, на уровне пикселей трудно описать, как выглядит колесо. Колесо имеет простую геометрическую форму, но распознавание его изображения может быть осложнено отбрасыванием теней, блеском солнца на металлических деталях автомобиля, наличием щитка, защищающего колесо от грязи, или объектов на переднем плане, частично загораживающих колесо, и т. д.

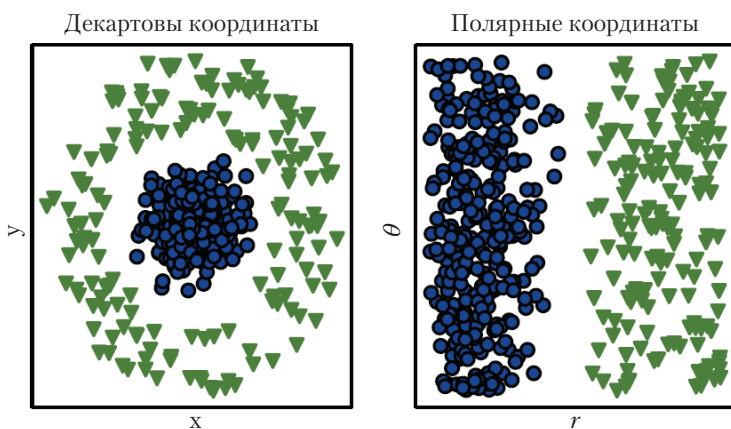


Рис. 1.1 ❖ Пример различных представлений: предположим, что требуется разделить две категории данных, проведя прямую на диаграмме рассеяния. На левом рисунке данные представлены в декартовых координатах, и задача неразрешима. На правом рисунке те же данные представлены в полярных координатах и разделяются вертикальной прямой (рисунок подготовлен совместно с Дэвидом Уорд-Фарли)

Одно из решений этой проблемы – воспользоваться машинным обучением не только для того, чтобы найти отображение представления на результат, но и чтобы определить само представление. Такой подход называется **обучением представлений**. На представлениях, полученных в ходе обучения, часто удается добиться гораздо более высокого качества, чем на представлениях, созданных вручную. К тому же это позволяет системам ИИ быстро адаптироваться к новым задачам при минималь-

ном вмешательстве человека. Для простой задачи алгоритм обучения представлений может найти хороший набор признаков за несколько минут, для сложных – за время от нескольких часов до нескольких месяцев. Проектирование признаков вручную для сложной задачи требует много времени и труда, на это могут уйти десятилетия работы всего сообщества исследователей.

Квинтэссенцией алгоритма обучения представлений является **автокодировщик**. Это комбинация функции **кодирования**, которая преобразует входные данные в другое представление, и функции **декодирования**, которая преобразует новое представление в исходный формат. Обучение автокодировщиков устроено так, чтобы при кодировании и обратном декодировании сохранялось максимально много информации, но чтобы при этом новое представление обладало различными полезными свойствами. Различные автокодировщики ориентированы на получение различных свойств.

При проектировании признаков или алгоритмов обучения признаков нашей целью обычно является выделение **факторов вариативности**, которые объясняют наблюдаемые данные. В этом контексте слово «фактор» означает просто источник влияния, а не «сомножитель». Зачастую факторы – это величины, не наблюдаемые непосредственно. Это могут быть ненаблюдаемые объекты или силы в физическом мире, оказывающие влияние на наблюдаемые величины. Это могут быть также умозрительные конструкции, дающие полезные упрощающие объяснения или логически выведенные причины наблюдаемых данных. Их можно представлять себе как концепции или абстракции, помогающие извлечь смысл из данных, характеризующихся высокой вариативностью. В случае анализа записи речи к факторам вариативности относятся возраст и пол говорящего, акцент и произносимые слова. В случае анализа изображения автомобиля факторами вариативности являются положение машины, ее цвет, а также высота солнца над горизонтом и его яркость.

Источник трудностей в целом ряде практических приложений искусственного интеллекта – тот факт, что многие факторы вариативности оказывают влияние абсолютно на все данные, доступные нашему наблюдению. Отдельные пиксели изображения красного автомобиля ночью могут быть очень близки к черному цвету. Форма силуэта автомобиля зависит от угла зрения. В большинстве приложений требуется *разделить* факторы вариативности и отбросить те, что нам не интересны.

Разумеется, может оказаться очень трудно выделить такие высокоуровневые абстрактные признаки из исходных данных. Многие факторы вариативности, к примеру акцент говорящего, можно идентифицировать, только если наличествует очень глубокое, приближающееся к человеческому понимание природы данных. Но раз получить представление почти так же трудно, как решить исходную задачу, то, на первый взгляд, обучение представлений ничем не поможет.

Глубокое обучение решает эту центральную проблему обучения представлений, вводя представления, выражаемые в терминах других, более простых представлений. Глубокое обучение позволяет компьютеру строить сложные концепции из более простых. На рис. 1.2 показано, как в системе глубокого обучения можно представить концепцию изображения человека в виде комбинации более простых концепций – углов и контуров, – которые, в свою очередь, определены в терминах границ.

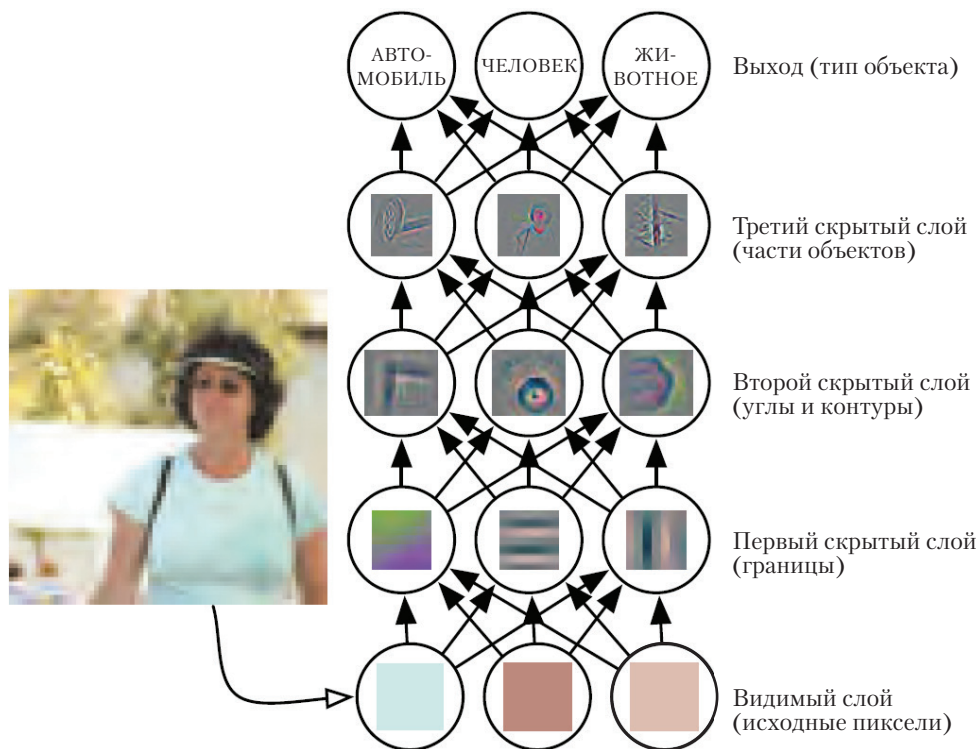


Рис. 1.2 ❖ Иллюстрация модели глубокого обучения. Компьютеру трудно понять смысл исходных данных, полученных от сенсоров, таких, например, как изображение, представленное в виде набора значений пикселей. Функция, отображающая множество пикселей в распознанный объект, очень сложна. Если подходить к задаче вычисления или обучения этого отображения в лоб, то она выглядит безнадежной. Глубокое обучение разрешает эту проблему, разбивая искомое сложное отображение на ряд более простых вложенных, каждое из которых описывается отдельным слоем модели. Входные данные представлены видимым слоем, он называется так, потому что содержит переменные, доступные наблюдению. За ним идет ряд скрытых слоев, которые извлекают из изображения все более и более абстрактные признаки. Слово «скрытый» означает, что значения, вырабатываемые этими слоями, не присутствуют в данных; сама модель должна определить, какие концепции полезны для объяснения связей в наблюдаемых данных. На рисунке показаны признаки, представленные каждым скрытым слоем. Зная исходные пиксели, первый слой легко может найти границы, для этого нужно лишь сравнить яркости соседних пикселей. Имея описание границ, выработанное первым скрытым слоем, второй скрытый слой находит углы и контуры в виде наборов границ. По этому описанию третий скрытый слой может распознать части конкретных объектов, представленные совокупностями контуров и углов определенного вида. Наконец, по описанию изображения в терминах частей объектов можно распознать сами объекты. Изображения взяты из работы Zeiler and Fergus (2014) с разрешения авторов

Типичным примером модели глубокого обучения является глубокая сеть прямого распространения, или **многослойный перцептрон** (МСП). Многослойный перцептрон – это просто математическая функция, отображающая множество входных значений на множество выходных. Эта функция является композицией нескольких более простых функций. Каждое применение одной математической функции можно рассматривать как новое представление входных данных.

Идея нахождения подходящего представления данных путем обучения – это лишь один взгляд на глубокое обучение. Другой взгляд состоит в том, что глубина позволяет обучать многошаговую компьютерную программу. Каждый слой представления можно мыслить себе как состояние памяти компьютера после параллельного выполнения очередного набора инструкций. Чем больше глубина сети, тем больше инструкций она может выполнить последовательно. Последовательное выполнение инструкций расширяет возможности, поскольку более поздние инструкции могут обращаться к результатам выполнения предыдущих.

При таком взгляде на глубокое обучение не всякая информация, используемая для активации слоев, обязательно кодирует факторы вариативности, объясняющие входные данные. В представлении хранится также вспомогательная информация о состоянии, помогающая выполнить программу, способную извлекать смысл из данных. Эту информацию можно уподобить счетчику или указателю в традиционной компьютерной программе. Она не имеет никакого отношения к содержанию входных данных, но помогает модели в организации их обработки.

Есть два основных способа измерить глубину модели. Первый оценивает архитектуру на основе числа последовательных инструкций, которые необходимо выполнить. Можно считать, что это длина самого длинного пути в графе, описывающем вычисление каждого выхода модели по ее входам. Как у двух эквивалентных компьютерных программ могут быть разные длины пути в зависимости от языка, на котором они написаны, так и одна и та же функциональность может быть изображена графами с разной длиной пути в зависимости от того, какие функции допускаются в качестве шагов. На рис. 1.3 показано, как выбор языка может дать разные результаты измерений для одной и той же архитектуры.

При другом подходе, используемом в глубоких вероятностных моделях, глубиной модели считается не глубина графа вычислений, а глубина графа, описывающего связи концепций. В этом случае граф вычислений, выполняемых для вычисления представления каждой концепции, может быть гораздо глубже, чем граф самих концепций. Связано это с тем, что понятие системы о простых концепциях можно уточнять, располагая информацией о более сложных. Например, система ИИ, наблюдающая изображение лица, на котором один глаз находится в тени, первоначально может распознать только один глаз. Но, обнаружив присутствие лица, система может заключить, что должен быть и второй глаз. В таком случае граф концепций содержит только два слоя – для глаз и для лиц, тогда как граф вычислений содержит $2n$ слоев, если мы n раз уточняем оценку каждой концепции при известной информации о второй.

Поскольку не всегда ясно, какой из двух подходов – глубина графа вычислений или глубина графа вероятностной модели – более релевантен, и поскольку разные

люди по-разному выбирают наборы примитивных элементов, из которых строятся графы, не существует единственно правильного значения глубины архитектуры, как не существует единственно правильной длины компьютерной программы. И нет общего мнения о том, какой должна быть глубина, чтобы модель можно было считать «глубокой». Однако можно все-таки сказать, что глубокое обучение – это наука о моделях, в которых уровень композиции обученных функций или обученных концепций выше, чем в традиционном машинном обучении.

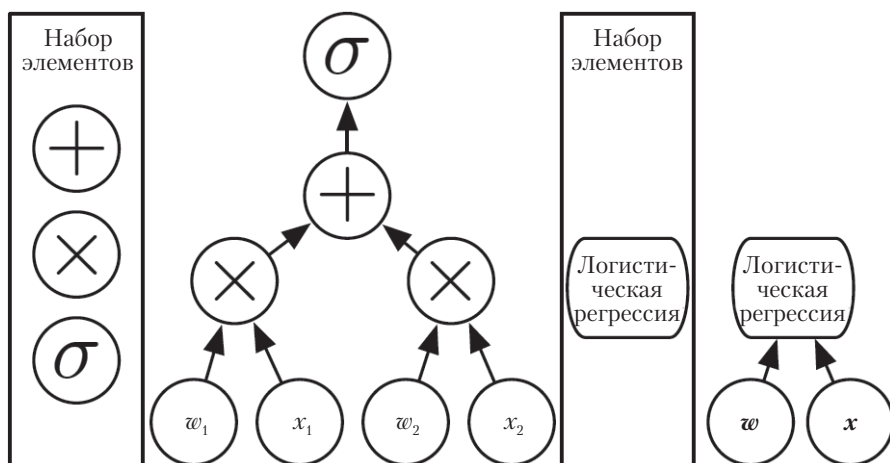


Рис. 1.3 ❖ Иллюстрация графов вычислений, переводящих вход в выход; в каждом узле выполняется некоторая операция. Глубиной считается длина самого длинного пути от входа к выходу, она зависит от определения допустимого шага вычисления. Оба графа описывают выход модели логистической регрессии $\sigma(\mathbf{w}^T \mathbf{x})$, где σ – логистическая сигмоида. Если в качестве элементов языка используются сложение, умножение и логистические сигмоиды, то глубина модели равна 3. Если же логистическая регрессия сама считается элементом языка, то глубина равна 1

Итак, глубокое обучение – тема этой книги – один из подходов к ИИ. Конкретно, это вид машинного обучения – методики, которая позволяет компьютерной системе совершенствоваться по мере накопления опыта и данных. Мы твердо убеждены, что машинное обучение – единственный жизнеспособный подход к построению систем ИИ, которые могут функционировать в сложных окружающих условиях. Глубокое обучение – это частный случай машинного обучения, позволяющий достичь большей эффективности и гибкости за счет представления мира в виде иерархии вложенных концепций, в которой каждая концепция определяется в терминах более простых концепций, а более абстрактные представления вычисляются в терминах менее абстрактных. На рис. 1.4 показано соотношение между разными отраслями ИИ, а на рис. 1.5 – высокоуровневое описание каждого подхода.



Рис. 1.4 ❖ На этой диаграмме Венна показано, что глубокое обучение – частный случай обучения представлений, которое, в свою очередь, является частным случаем машинного обучения, используемого во многих, но не во всех подходах к ИИ. На каждой части диаграммы Венна приведен пример технологии ИИ

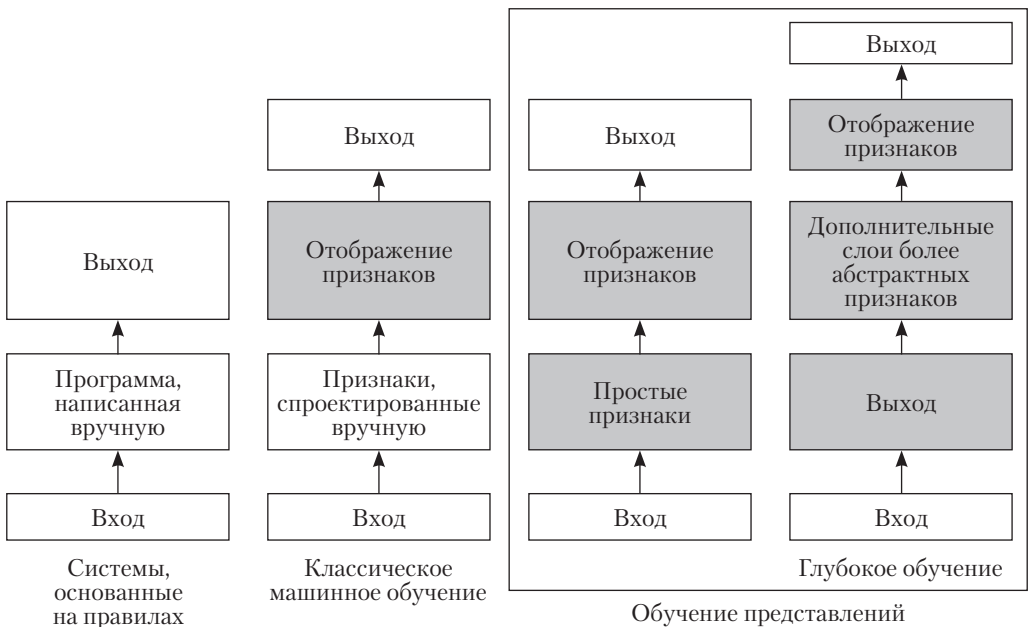


Рис. 1.5 ❖ Связь различных частей системы ИИ между собой в рамках разных подходов к ИИ. Серым цветом показаны компоненты, способные обучаться на данных

1.1. На кого ориентирована эта книга

Книга будет полезна разным читателям, но мы писали ее, имея в виду две целевые аудитории. Во-первых, это студенты университетов (как младших, так и старших курсов), изучающие машинное обучение, в том числе и те, кто начинает строить карьеру в области машинного обучения и искусственного интеллекта. Во-вторых, это разработчики ПО, которые не изучали машинное обучение или статистику, но хотят быстро освоить эти дисциплины и приступить к использованию глубокого обучения в своем продукте или платформе. Глубокое обучение доказало полезность во многих направлениях ПО, в т. ч. компьютерном зрении, распознавании речи и аудиозаписей, обработке естественных языков, робототехнике, биоинформатике и химии, видеоиграх, поисковых системах, интернет-рекламе и финансах.

Книга разделена на три части, чтобы лучше удовлетворить потребности разных категорий читателей. В части I излагается базовый математический аппарат и дается введение в концепции машинного обучения. В части II описаны самые известные алгоритмы глубокого обучения, которые можно считать сформировавшимися технологиями. Часть III посвящена более спорным идеям, которые многие считают важными для будущих исследований в области глубокого обучения.

Читатель может без ущерба для понимания пропускать части, не отвечающие его интересам или подготовке. Например, знакомые с линейной алгеброй, теорией вероятностей и фундаментальными концепциями машинного обучения могут пропустить часть I, а желающим реализовать работающую систему нет необходимости читать дальше части II. Чтобы читателю было проще понять, какие главы ему интересны, на рис. 1.6 показано, как устроена книга в целом.

Мы предполагаем, что читатели имеют подготовку в области информатики: знакомы с программированием, понимают, что такое производительность вычислений и теория сложности, знают начала математического анализа и некоторые положения и термины теории графов.

1.2. Исторические тенденции в машинном обучении

Понять глубокое обучение проще всего в историческом контексте. Мы не станем детально описывать историю глубокого обучения, а выделим несколько ключевых тенденций:

- у глубокого обучения долгая и богатая история, но оно фигурировало под разными названиями, отражающими различные философские воззрения, в связи с чем его популярность то усиливалась, то ослабевала;
- полезность глубокого обучения возросла, когда увеличился объем доступных обучающих данных;
- размер моделей глубокого обучения увеличивался по мере того, как совершенствовалась компьютерная инфраструктура (программная и аппаратная);
- со временем с помощью глубокого обучения удавалось решать все более сложные задачи со все большей точностью.

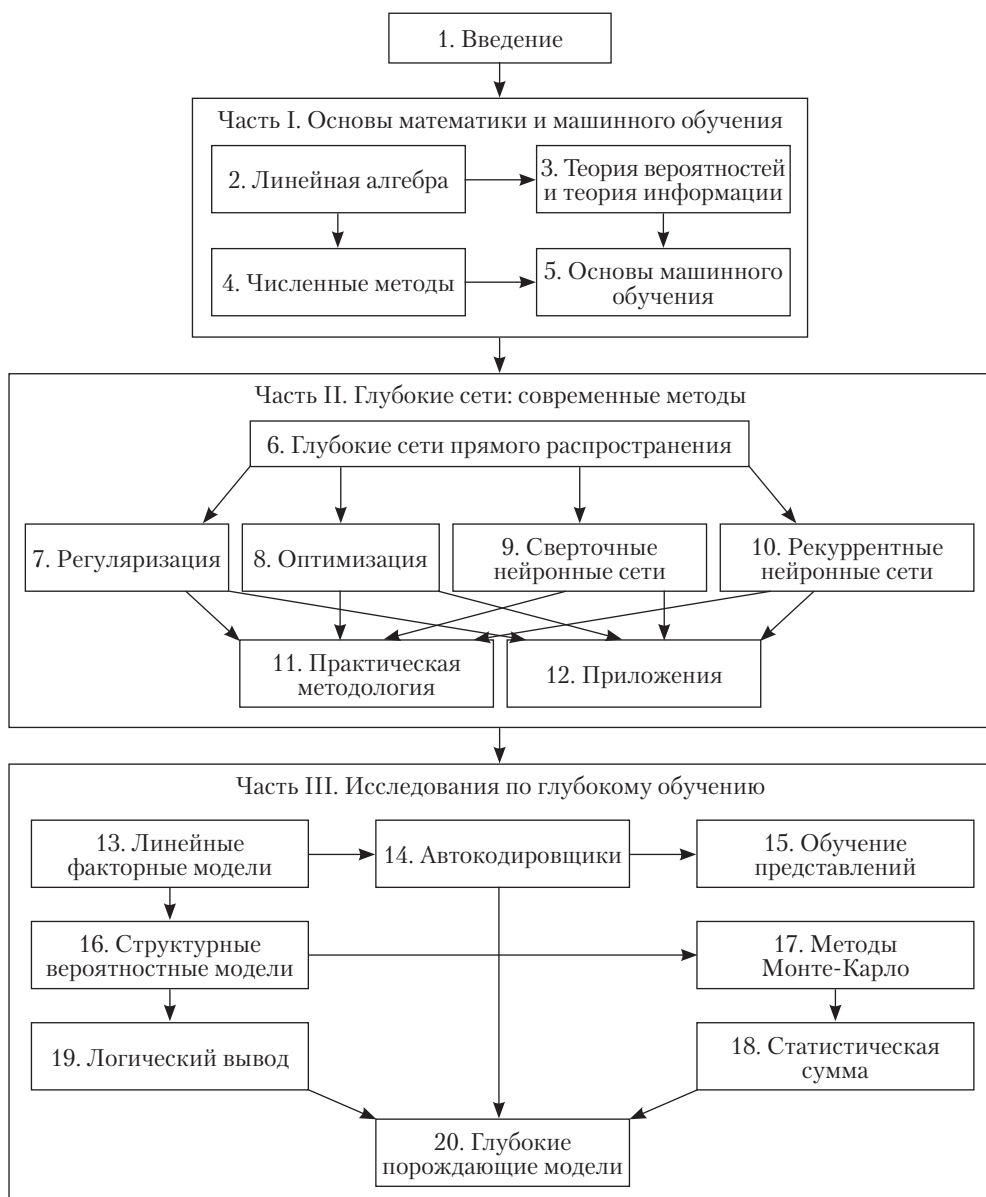


Рис. 1.6 ❖ Структура книги. Стрелки означают, что одна глава содержит материал, необходимый для понимания другой

1.2.1. Нейронные сети: разные названия и переменчивая фортуна

Мы полагаем, что многие читатели слышали о глубоком обучении как о сулящей чудеса новой технологии и удивлены, встретив слово «история» в применении к только зарождающейся дисциплине. Но в действительности глубокое обучение возникло еще в 1940-х годах. Оно кажется новым лишь потому, что в течение нескольких

лет, предшествующих нынешнему всплеску популярности, прозябало в тени, а также потому, что названия менялись, и только недавно эта дисциплина стала называться «глубоким обучением». Прежние названия отражали вес разных исследователей в научных кругах и разные точки зрения на предмет.

Изложение полной истории глубокого обучения выходит за рамки этого учебника. Но кое-какие базовые сведения помогут лучше понять его смысл. Если отвлечься от деталей, то было три волны разработок: в 1940–1960-х годах глубокое обучение было известно под названием **кибернетики**, в 1980–1990-х – как **коннекционизм**, а в современной инкарнации – под нынешним названием – оно возродилось в 2006 году. Количественная картина показана на рис. 1.7.

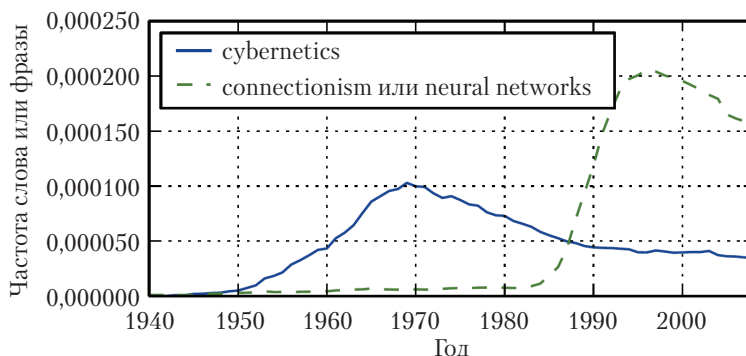


Рис. 1.7 ❖ Две из трех исторических волн исследований по искусственным нейронным сетям, оцененные по частоте фраз «cybernetics» и «connectionism или neural networks» согласно Google Books (третья волна началась недавно и еще не отражена). Первая волна, связанная с кибернетикой, приходится на 1940–1960-е годы, когда разрабатывались теории биологического обучения (McCulloch and Pitts, 1943; Hebb, 1949) и были реализованы первые модели, в частности перцептрон (Rosenblatt, 1958), позволявшие обучить один нейрон. Вторая волна периода 1980–1995 гг. связана с коннекционистским подходом, когда метод обратного распространения (Rumelhart et al., 1986a) был применен к обучению нейронной сети с одним или двумя скрытыми слоями. Третья волна, глубокое обучение, началась примерно в 2006 году (Hinton et al., 2006; Bengio et al., 2007; Ranzato et al., 2007a) и только теперь – в 2016 году – описывается в виде книги. Книги, посвященные двум другим волнам, также вышли гораздо позже активизации исследований в соответствующей области

Некоторые из самых ранних алгоритмов обучения сегодня мы назвали бы компьютерными моделями биологического обучения, т. е. процессов, которые происходят или могли бы происходить в мозге. Поэтому одно из прежних названий глубокого обучения – искусственные нейронные сети (ИНС). Им соответствует взгляд на модели глубокого обучения как на инженерные системы, устроенные по образцу биологического мозга (человека или животного). Но хотя были попытки использовать нейронные сети, применяемые в машинном обучении, чтобы понять, как функционирует мозг (Hinton and Shallice, 1991), в общем случае при их проектировании не ставилась задача создать реалистическую модель биологической функции. Нейронный подход к глубокому обучению основан на двух главных идеях. Во-первых, мозг – доказатель-

ный пример того, что разумное поведение возможно. Поэтому концептуально прямой путь к построению искусственного интеллекта состоит в том, чтобы проанализировать с вычислительной точки зрения принципы работы мозга и воспроизвести его функциональность. Во-вторых, вообще было бы очень интересно понять, как работает мозг и какие принципы лежат в основе разума человека, поэтому модели машинного обучения, проливающие свет на эти фундаментальные научные вопросы, полезны вне зависимости от их применимости к конкретным инженерным задачам.

Современный термин «глубокое обучение» выходит за рамки нейробиологического взгляда на модели машинного обучения. В нем заложен более общий принцип обучения нескольких уровней композиции, применимый к системам машинного обучения, не обязательно устроенным по примеру нейронов.

Предтечами современного глубокого обучения были простые линейные модели на основе нейробиологических аналогий. Они принимали множество n входных значений $x_1, \dots, x_n$ и ассоциировали с ними выход y . В ходе обучения модель должна была найти веса $w_1, \dots, w_n$ и вычислить выход в виде $f(\mathbf{x}, \mathbf{w}) = x_1 w_1 + \dots + x_n w_n$. Эта первая волна нейронных сетей известна под названием кибернетики (рис. 1.7).

Нейрон Маккаллока–Питтса (McCulloch and Pitts, 1943) был ранней моделью функционирования мозга. Эта линейная модель могла распознавать две категории выходов, проверяя, является значение $f(\mathbf{x}, \mathbf{w})$ положительным или отрицательным. Конечно, чтобы модель соответствовала желаемому определению категорий, нужно было правильно подобрать веса. Веса задавал человек. В 1950-е годы был изобретен перцептрон (Rosenblatt, 1958, 1962) – первая модель, которая могла в процессе обучения находить веса, определяющие категории, имея примеры входных данных из каждой категории. Модель **адаптивного линейного элемента** (ADALINE), относящаяся примерно к тому же времени, просто возвращала само значение $f(\mathbf{x})$ для предсказания вещественного числа (Widrow and Hoff, 1960) и также могла обучаться предсказанию чисел на данных.

Эти простые алгоритмы обучения оказали заметное влияние на современный ландшафт машинного обучения. Алгоритм обучения, использованный для подбора весов в модели ADALINE, был частным случаем алгоритма **стохастического градиентного спуска**. Его немного модифицированные варианты и по сей день остаются основными алгоритмами моделей глубокого обучения.

Модели на основе функции $f(\mathbf{x}, \mathbf{w})$, используемые в перцептроне и ADALINE, называются **линейными моделями**. Они до сих пор относятся к числу наиболее распространенных моделей машинного обучения, хотя часто обучаются иначе, чем было принято раньше.

У линейных моделей много ограничений. Самое известное состоит в невозможности обучить функцию XOR, для которой $f([0, 1], \mathbf{w}) = 1$ и $f([1, 0], \mathbf{w}) = 1$, но $f([1, 1], \mathbf{w}) = 0$ и $f([0, 0], \mathbf{w}) = 0$. Критики, отмечавшие эти изъяны линейных моделей, вообще возражали против обучения, основанного на биологических аналогиях (Minsky and Papert, 1969). Это стало первым серьезным ударом по популярности нейронных сетей. В настоящее время нейробиология рассматривается как важный источник идей для исследований в области глубокого обучения, но уже не занимает доминирующих позиций.

Главная причина снижения роли нейробиологии в исследованиях по глубокому обучению – тот факт, что у нас попросту не хватает информации о мозге, чтобы использовать ее в качестве образца и руководства к действию. Чтобы по-настоящему

понять алгоритмы работы мозга, нужно было бы одновременно наблюдать за активностью тысяч (как минимум) взаимосвязанных нейронов. Не имея такой возможности, мы далеки от понимания даже самых простых и хорошо изученных областей мозга (Olshausen and Field, 2005).

Нейробиология дала надежду на то, что один алгоритм глубокого обучения сможет решить много разных задач. Нейробиологи выяснили, что хорьки могут «видеть» с помощью области мозга, отвечающей за обработку слуховой информации, если перенаправить нервы из глаз в слуховую кору (Von Melchner et al., 2000). Это наводит на мысль, что значительная часть мозга млекопитающих, возможно, использует единый алгоритм для решения большинства задач. До появления этой гипотезы исследования по машинному обучению были более фрагментированы: обработкой естественных языков, компьютерным зрением, планированием движения и распознаванием речи занимались разные группы ученых. Эти сообщества и по сей день разделены, но ученые, работающие в области глубокого обучения, зачастую занимаются многими или даже всеми этими предметами.

Мы можем почерпнуть из нейробиологии кое-какие соображения. Основная идея, навеянная осмыслением работы мозга, – наличие большого числа вычислительных блоков, которые обретают разум только в результате взаимодействий. Неокогнитрон (Fukushima, 1980) предложил архитектуру модели, эффективную для обработки изображений. Идея сложилась под влиянием структуры зрительной системы млекопитающих и впоследствии легла в основу современной сверточной сети (LeCun et al., 1998b), с которой мы познакомимся в разделе 9.10. Большинство современных нейронных сетей основано на модели нейрона, которая называется **блоком линейной ректификации**. Оригинальная модель когнитрона (Fukushima, 1975) была более сложной, основанной на наших знаниях о работе мозга. В современной упрощенной модели объединены различные точки зрения; в работах Nair and Hinton (2010) и Glorot et al. (2011a) отмечено влияние нейробиологии, а в работе Jarrett et al. – скорее, инженерных дисциплин. Каким бы важным источником идей ни была нейробиология, считать, что от нее нельзя отклониться ни на шаг, вовсе необязательно. Мы знаем, что настоящие нейроны вычисляют совсем не те функции, что блоки линейной ректификации, но большее приближение к реальности пока не привело к повышению качества машинного обучения. Кроме того, хотя нейробиология легла в основу нескольких успешных архитектур нейронных сетей, мы до сих пор знаем о биологическом обучении недостаточно, чтобы полнее использовать нейробиологию для построения *алгоритмов обучения* этих архитектур.

В социальных сетях часто подчеркивают сходство между глубоким обучением и мозгом. Да, действительно, исследователи, занимающиеся глубоким обучением, чаще говорят о влиянии аналогий с мозгом на свою работу, чем те, кто занимается другими методами машинного обучения, например ядерными методами или байесовской статистикой. Но не следует думать, что цель глубокого обучения – попытка имитировать мозг. Современное глубокое обучение черпает идеи из разных дисциплин, особенно из таких отраслей прикладной математики, как линейная алгебра, теория вероятностей, теория информации и численная оптимизация. Некоторые ученые считают нейробиологию важным источником идей, другие не упоминают ее вовсе.

Отметим, что попытки понять, как работает мозг на алгоритмическом уровне, не прекращаются. Эта область исследований, известная под названием «вычислительная нейробиология», не совпадает с глубоким обучением. Нередко ученые

переходят из одной области в другую. Предмет глубокого обучения – построение компьютерных систем, способных успешно решать задачи, требующие интеллекта, а предмет вычислительной нейробиологии – построение более точных моделей работы мозга.

В 1980-е годы поднялась вторая волна исследований по нейронным сетям, вызванная главным образом движением под названием **коннекционизм**, или **параллельная распределенная обработка** (Rumelhart et al., 1986c; McClelland et al., 1995). Коннекционизм возник в контексте когнитивистики – междисциплинарного подхода к пониманию процесса познания, объединяющего несколько разных уровней анализа. В начале 1980-х годов большинство когнитивистов изучало модели принятия решений путем манипулирования символами (symbolic reasoning). Несмотря на популярность символических моделей, трудно было объяснить, как мозг мог бы реализовать их с помощью нейронов. Коннекционисты начали изучать модели познания, которые допускали реализацию на основе нейронов (Touretzky and Minton, 1985), возродив многие идеи психолога Дональда Хебба, высказанные в 1940-х годах (Hebb, 1949).

Центральная идея коннекционизма состоит в том, что при наличии большого количества вычислительных блоков, объединенных в сеть, удастся достичь разумного поведения. Эта идея относится в равной мере к нейронам в биологических нервных системах и к скрытым блокам в компьютерных моделях.

Движение коннекционизма породило несколько ключевых концепций, которые и по сей день играют важнейшую роль в глубоком обучении.

Одна из них – **распределенное представление** (Hinton et al., 1986). Идея в том, что каждый вход системы следует представлять многими признаками, а каждый признак должен участвовать в представлении многих возможных входов. Пусть, например, имеется зрительная система, способная распознавать легковые автомобили, грузовики и птиц, причем объекты могут быть красного, зеленого или синего цвета. Один из способов представления таких входов – завести отдельный нейрон или скрытый блок для активации каждой из девяти возможных комбинаций: красный грузовик, красная легковушка, красная птица, зеленый грузовик и т. д. Тогда потребуется девять нейронов, и каждый нейрон необходимо независимо обучить концепциям цвета и типа объекта. Улучшить ситуацию можно, воспользовавшись распределенным представлением, в котором три нейрона описывают цвет, а еще три – тип объекта. Тогда понадобится всего шесть нейронов, и нейрон, отвечающий за красное, можно обучить на изображениях легковушек, грузовиков и птиц, а не только на изображениях объектов одного типа. Концепция распределенного представления является центральной в этой книге и подробно рассматривается в главе 15.

Еще одним крупным достижением коннекционистов стали успешное использование обратного распространения для обучения глубоких нейронных сетей с внутренними представлениями и популяризация алгоритма обратного распространения (Rumelhart et al., 1986a; LeCun, 1987). Его популярность то возрастала, то убывала, но на данный момент это преобладающий подход к обучению глубоких моделей.

1990-е годы стали временем важных достижений в моделировании последовательностей с помощью нейронных сетей. В работах Hochreiter (1991) и Bengio et al. (1994) сформулирован ряд фундаментальных математических трудностей моделирования длинных последовательностей (см. раздел 10.7). В работе Hochreiter and Schmidhuber (1997) введено понятие сетей с долгой краткосрочной памятью (long short-term me-

могу – LSTM) для разрешения некоторых из описанных трудностей. Сегодня LSTM-сети широко используются во многих задачах моделирования последовательностей, в т. ч. для обработки естественных языков в Google.

Вторая волна работ по нейронным сетям продолжалась до середины 1990-х годов. Но компании, специализирующиеся на нейронных сетях и других технологиях ИИ, стали давать чрезмерно амбициозные обещания в попытках привлечь инвестиции. Когда ИИ не оправдал этих неразумных надежд, инвесторы испытали разочарование. В то же время имел место заметный прогресс в других областях машинного обучения. Ядерные методы (Boser et al., 1992; Cortes and Vapnik, 1995; Schölkopf et al., 1999) и графические модели (Jordan, 1998) позволили достичь хороших результатов при решении многих важных задач. В совокупности эти два фактора привели к спаду интереса к нейронным сетям, который продолжался до 2007 года.

В это время нейронные сети по-прежнему показывали впечатляющее качество на некоторых задачах (LeCun et al., 1998b; Bengio et al., 2001). Канадский институт перспективных исследований (Canadian Institute for Advanced Research – CIFAR) помог нейронным сетям остаться на плаву, профинансировав исследовательскую программу нейронных вычислений и адаптивного восприятия (Neural Computation and Adaptive Perception – NCAP). В рамках этой программы объединились группы Джеффри Хинтона из Торонтского университета, Иошуа Бенджио из Монреальского университета и Янна Лекуна из Нью-Йоркского университета. Мультидисциплинарная исследовательская программа CIFAR NCAP включала также нейробиологов и специалистов по человеческому и компьютерному зрению.

Тогда сложилось общее мнение, что обучить глубокие сети очень трудно. Теперь мы знаем, что алгоритмы, существовавшие начиная с 1980-х годов, работают отлично, но это не было очевидно до 2006 года. Причина, наверное, в том, что с вычислительной точки зрения эти алгоритмы очень накладны, поэтому было невозможно экспериментировать с ними на имевшемся тогда оборудовании.

Третья волна работ по нейронным сетям началась с прорыва в 2006 году. Джеффри Хинтон показал, что так называемые **глубокие сети доверия** можно эффективно обучать с помощью стратегии жадного послойного предобучения (Hinton et al., 2006), которую мы подробно опишем в разделе 15.1. Другие аффилированные с CIFAR исследовательские группы быстро показали, что ту же стратегию можно использовать для обучения многих других видов глубоких сетей (Bengio et al., 2007; Ranzato et al., 2007a), и систематически улучшали степень обобщения на тестовых примерах. Благодаря этой волне исследований в обиход вошел термин «глубокое обучение», подчеркивающий, что теперь можно обучать более глубокие нейронные сети, чем раньше, и привлекающий внимание к теоретической важности глубины (Bengio and LeCun, 2007; Delalleau and Bengio, 2011; Pascanu et al., 2014a; Montufar et al., 2014). В то время глубокие нейронные сети превзошли конкурирующие системы ИИ – как основанные на других технологиях машинного обучения, так и спроектированные вручную. Третья волна популярности нейронных сетей продолжается и во время написания этой книги, хотя фокус исследований значительно сместился. Поначалу в центре внимания находились методы обучения без учителя и способность глубоких моделей, обученных на небольших наборах данных, к обобщению. А теперь больший интерес вызывают гораздо более старые алгоритмы обучения с учителем и возможность задействовать большие размеченные наборы данных при обучении глубоких моделей.

1.2.2. Увеличение размера набора данных

Может возникнуть вопрос, почему лишь недавно была осознана роль глубокого обучения как ключевой технологии, хотя первые эксперименты с искусственными нейронными сетями были проведены еще в 1950-х годах. Глубинное обучение успешно применяется в коммерческих приложениях, начиная с 1990-х, но часто его рассматривали не как технологию, а как искусство – как нечто такое, что подвластно только экспертам. Так было до недавнего времени. Действительно, чтобы добиться хорошего качества от алгоритма глубокого обучения, нужен некоторый навык. Но, к счастью, потребность в таком навыке снижается по мере увеличения объема обучающих данных. Алгоритмы обучения, по качеству приближающиеся к возможностям человека при решении сложных задач, остались почти такими же, как алгоритмы, с трудом справлявшиеся с игрушечными задачами в 1980-х, но модели, которые мы с их помощью обучаем, претерпели изменения, позволившие упростить обучение очень глубоких архитектур. Самое важное новшество заключается в том, что сегодня мы можем предоставить этим алгоритмам необходимые для успеха ресурсы. На рис. 1.8 показано изменение эталонных наборов данных со временем. Эта тенденция обусловлена возрастающей цифровизацией общества. Чем активнее применяются компьютеры, тем больше записей о том, что мы делаем. А поскольку компьютеры объединяются в сети, становится проще централизованно хранить эти записи и построить из них набор данных, подходящий для машинного обучения. С наступлением эры «больших данных» машинное обучение значительно упростилось, поскольку ключевая проблема статистического оценивания – высокое качество обобщения на новые данные после обучения на небольшом количестве примеров – теперь далеко не так актуальна. В 2016 году действует грубое эвристическое правило: алгоритм глубокого обучения с учителем достигает приемлемого качества при наличии примерно 5000 помеченных примеров на категорию и оказывается сопоставим или даже превосходит человека, если обучается на наборе данных, содержащем не менее 10 миллионов помеченных примеров. Как добиться успеха при работе с наборами данных меньшего размера – важная область исследований, и акцент в ней делается на том, чтобы воспользоваться преимуществами большого количества непомеченных примеров, применяя обучение без учителя или с частичным привлечением учителя.

1.2.3. Увеличение размера моделей

Еще одна причина нынешнего роста популярности нейронных сетей после сравнительно скромных успехов в 1980-е годы – наличие вычислительных ресурсов, достаточных для работы с гораздо более крупными моделями. Один из главных выводов коннекционизма состоит в том, что животные проявляют интеллект, когда много нейронов работает совместно. Один нейрон или небольшой их набор не принесет особой пользы.

Плотность соединений между биологическими нейронами не слишком велика. Как видно по рис. 1.10, в наших моделях машинного обучения число соединений в расчете на один нейрон примерно на порядок выше, чем даже в мозгу млекопитающих, и такое положение существует уже несколько десятков лет.

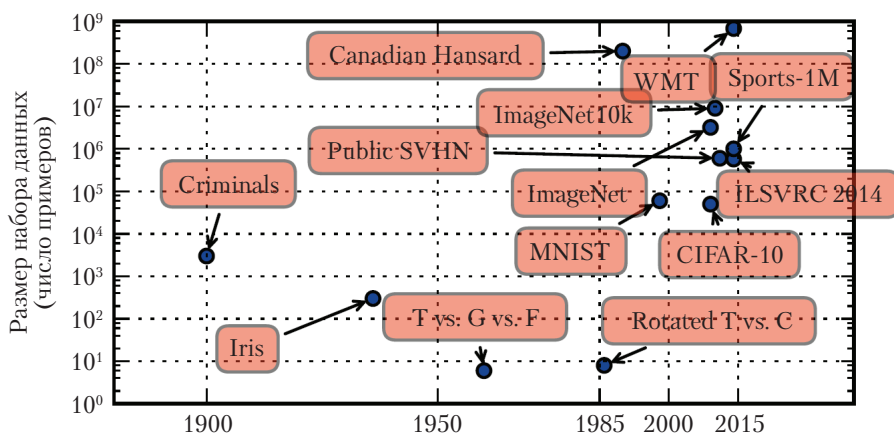


Рис. 1.8 ❖ Увеличение размера набора данных со временем. В начале 1900-х годов статистики изучали наборы данных, содержащие от сотен до тысяч вручную подготовленных измерений (Garson, 1900; Gosset, 1908; Anderson, 1935; Fisher, 1936). В период между 1950-ми и 1980-ми пионеры биокомпьютерного зрения зачастую работали с небольшими синтетическими наборами данных, например растровыми изображениями букв низкого разрешения, специально спроектированными так, чтобы снизить стоимость вычислений и продемонстрировать, что нейронные сети можно обучить функциям специального вида (Widrow and Hoff, 1960; Rumelhart et al., 1986b). В 1980-е и 1990-е машинное обучение стало в большей степени статистическим, а наборы данных уже насчитывали десятки тысяч примеров, как, например, набор MNIST (рис. 1.9) отсканированных рукописных цифр (LeCun et al., 1998b). В первом десятилетии XXI века продолжали создавать более изощренные наборы данных того же размера, например CIFAR-10 (Krizhevsky and Hinton, 2009). В конце этого периода и в первой половине 2010-х появление гораздо больших наборов данных, содержащих от сотен тысяч до десятков миллионов примеров, полностью изменило представление о возможностях глубокого обучения. К таким наборам относится общедоступный набор номеров домов (Street View House Numbers) (Netzer et al., 2011), различные варианты набора ImageNet (Deng et al., 2009, 2010a; Russakovsky et al., 2014a) и набор Sports-1M (Karpthy et al., 2014). В верхней части диаграммы мы видим, что наборы переведенных предложений, например набор, построенный IBM по официальным отчетам о заседаниях канадского парламента (Brown et al., 1990), и набор WMT 2014 переводов с английского на французский (Schwenk, 2014), по размеру намного превосходят большинство остальных наборов

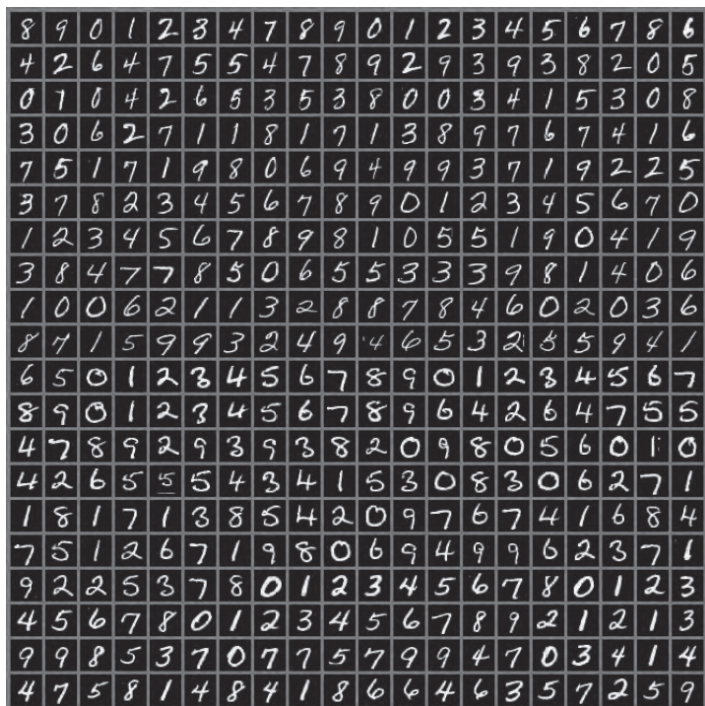


Рис. 1.9 ❖ Несколько примеров из набора данных MNIST. Акроним «NIST» означает «National Institute of Standards and Technology» (Национальный институт стандартов и технологий) – учреждение, первоначально собравшее эти данные. А буква «М» означает «модифицированный», поскольку данные были подвергнуты предварительной обработке, чтобы упростить применение алгоритмов машинного обучения. Набор данных MNIST содержит отсканированные изображения рукописных цифр и ассоциированные с ними метки, описывающие, какая цифра от 0 до 9 изображена. Это одна из самых простых задач классификации, поэтому набор широко используется для тестирования в исследованиях по глубокому обучению. Он сохраняет популярность, хотя для современных методов задача не представляет труда. Джеффри Хинтон назвал его «дрозофилой машинного обучения», имея в виду, что он позволяет специалистам по машинному обучению исследовать свои алгоритмы в контролируемых лабораторных условиях, как биологи изучают фруктовых мушек

Если говорить об общем числе нейронов, то до недавнего времени нейронные сети были на удивление малы (рис. 1.11). После добавления скрытых блоков размер искусственных нейронных сетей удваивался в среднем каждые 2,4 года, чему способствовало появление более быстрых компьютеров с большим объемом памяти, а также наличие крупных наборов данных. Чем больше сеть, тем выше ее точность на более сложных задачах. Эта тенденция прослеживается на протяжении десятилетий. Если темпы масштабирования не ускорятся в результате внедрения новых технологий, то искусственная нейронная сеть сравняется с человеческим мозгом по числу нейронов

не раньше 2050-х годов. Биологические нейроны могут представлять более сложные функции, чем современные искусственные, поэтому биологические нейронные сети могут оказаться даже больше, чем показано на диаграмме.

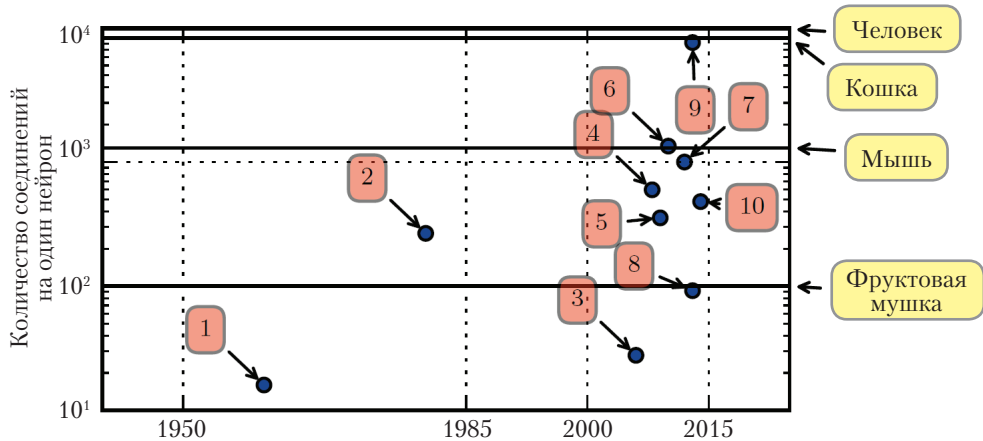


Рис. 1.10 ❖ Рост количества соединений в расчете на один нейрон со временем. Первоначально число соединений между нейронами в искусственной нейронной сети было ограничено возможностями оборудования. Сегодня оно в основном является параметром проектирования. В некоторых нейронных сетях число соединений почти такое же, как у кошки, а сети с числом соединений, как у мелких млекопитающих типа мыши, встречаются сплошь и рядом. Даже в мозге человека число соединений на нейрон нельзя назвать заоблачным. Размеры биологических нейронных сетей взяты из Википедии (2015)

1. Адаптивный линейный элемент (Widrow and Hoff, 1960)
2. Неокогнитрон (Fukushima, 1980)
3. GPU-ускоренная сверточная сеть (Chellapilla et al., 2006)
4. Глубокая машина Больцмана (Salakhutdinov and Hinton, 2009a)
5. Сверточная сеть без учителя (Jarrett et al., 2009)
6. GPU-ускоренный многослойный перцептрон (Ciresan et al., 2010)
7. Распределенный автокодировщик (Le et al., 2012)
8. Сверточная сеть с несколькими GPU (Krizhevsky et al., 2012)
9. Сверточная сеть без учителя на компьютерах типа COTS HPC (Coates et al., 2013)
10. GoogLeNet (Szegedy et al., 2014a)

В ретроспективе не кажется удивительным, что нейронные сети с числом нейронов меньше, чем у пиявки, не могли справиться с решением сложных задач искусственного интеллекта. Даже нынешние сети, которые мы считаем очень большими с вычислительной точки зрения, меньше нервной системы сравнительно примитивных позвоночных животных, например лягушек.

Увеличение размера моделей вследствие доступности более быстрых процессоров, GPU общего назначения (см. раздел 12.1.2), более быстрых сетей и более совершенной инфраструктуры распределенных вычислений — одна из самых важных тенденций в истории глубокого обучения. Ожидается, что она продолжится и в будущем.

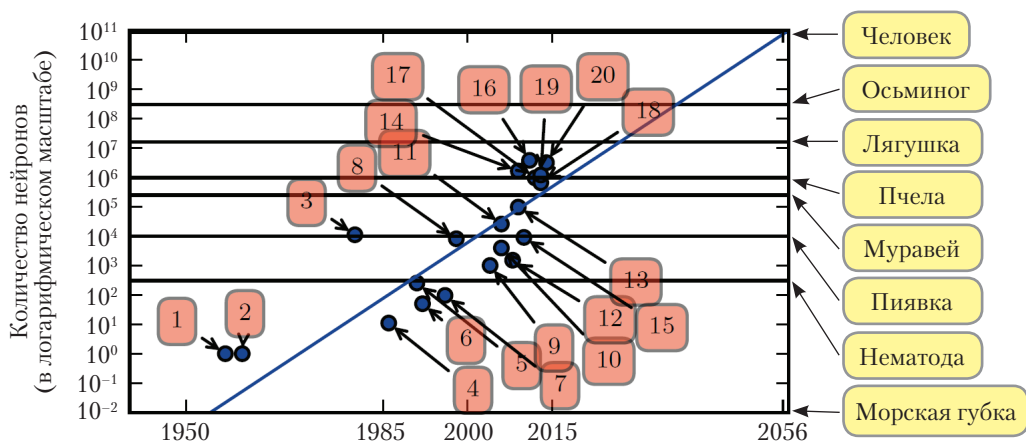


Рис. 1.11 ❖ Рост размера нейронной сети со временем. После добавления скрытых блоков размер нейронных сетей удваивался примерно каждые 2,4 года. Размеры биологических нейронных сетей взяты из Википедии (2015)

1. Перцептрон (Rosenblatt, 1958, 1962)
2. Адаптивный линейный элемент (Widrow and Hoff, 1960)
3. Неокогнитрон (Fukushima, 1980)
4. Ранняя сеть с обратным распространением (Rumelhart et al., 1986b)
5. Рекуррентная нейронная сеть для распознавания речи (Robinson and Fallside, 1991)
6. Многослойный перцептрон для распознавания речи (Bengio et al., 1991)
7. Сигмоидальная сеть доверия со средним полем (Saul et al., 1996)
8. LeNet-5 (LeCun et al., 1998b)
9. Нейронная эхо-сеть (Jaeger and Haas, 2004)
10. Глубокая сеть доверия (Hinton et al., 2006)
11. GPU-ускоренная сверточная сеть (Chellapilla et al., 2006)
12. Глубокая машина Больцмана (Salakhutdinov and Hinton, 2009a)
13. GPU-ускоренная сеть глубокого доверия (Raina et al., 2009)
14. Сверточная сеть без учителя (Jarrett et al., 2009)
15. GPU-ускоренный многослойный перцептрон (Ciresan et al., 2010)
16. Сеть OMP-1 (Coates and Ng, 2011)
17. Распределенный автокодировщик (Le et al., 2012)
18. Сверточная сеть с несколькими GPU (Krizhevsky et al., 2012)
19. Сверточная сеть без учителя на компьютерах типа COTS HPC (Coates et al., 2013)
20. GoogLeNet (Szegedy et al., 2014a)

1.2.4. Повышение точности и сложности и расширение круга задач

Начиная с 1980-х годов точность распознавания и прогнозирования глубоких моделей постоянно росла. И вместе с тем все разнообразнее становились задачи, которые удавалось решать с их помощью.

Самые первые глубокие модели использовались для распознавания отдельных объектов в кадрированных изображениях совсем небольшого размера (Rumelhart et al., 1986a). С тех пор размер изображений, которые можно было обработать с помощью нейронной сети, постепенно увеличивался. Современные сети распознавания объектов обрабатывают фотографии с высоким разрешением и не требуют кадрирования фотографии по месту расположения объекта (Krizhevsky et al., 2012). Кроме того, ранние сети умели распознавать только два вида объектов (а в некоторых случаях при-

сутствие или отсутствие объектов одного вида), тогда как типичная современная сеть распознает не менее 1000 категорий объектов. Самый крупный конкурс по распознаванию объектов – ImageNet Large Scale Visual Recognition Challenge (ILSVRC) – проводится каждый год. Переломным моментом, ознаменовавшим стремительный взлет глубокого обучения, стала победа с большим отрывом сверточной сети, которая участвовала впервые и сразу уменьшила частоту непопадания в первые пять (top-5 error rate) с 26,1 до 15,3% (Krizhevsky et al., 2012). Смысл этого показателя следующий: сверточная сеть порождала для каждого изображения ранжированный список возможных категорий, и правильная категория отсутствовала среди первых пяти элементов этого списка только в 15,3% тестовых примеров. С тех пор подобные конкурсы неизменно выигрывали сверточные сети, и на данный момент прогресс глубокого обучения позволил довести частоту непопадания в первые пять до 3,6% (рис. 1.12).

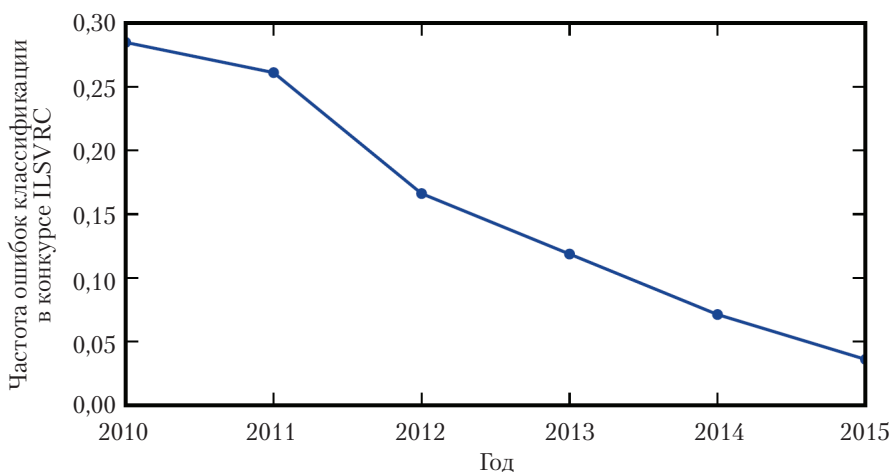


Рис. 1.12 ❖ Уменьшение частоты ошибок со временем. С тех пор как глубокие сети достигли масштаба, необходимого для участия в конкурсе ImageNet Large Scale Visual Recognition Challenge, они неизменно выигрывают его, с каждым разом демонстрируя все меньшую и меньшую частоту ошибок. Данные взяты из работ Russakovsky et al. (2014b) и He et al. (2015)

Глубокое обучение также оказало огромное влияние на распознавание речи. После прогресса, достигнутого на протяжении 1990-х годов, в качестве распознавания речи наступил застой. Применение глубокого обучения (Dahl et al., 2010; Deng et al., 2010b; Seide et al., 2011; Hinton et al., 2012a) привело к резкому уменьшению частоты ошибок, иногда аж наполовину. Мы вернемся к этой теме в разделе 12.3.

Глубокие сети добились также впечатляющих успехов в обнаружении пешеходов и сегментации изображений (Sermanet et al., 2013; Farabet et al., 2013; Couprie et al., 2013), а в задаче классификации дорожных знаков показали себя лучше человека (Ciresan et al., 2012).

Одновременно с повышением размера и точности глубоких сетей росла и сложность решаемых с их помощью задач. В работе Goodfellow et al. (2014d) показано, что нейронные сети можно научить распознаванию целых последовательностей символов в изображении, а не только идентификации одиночного объекта. Ранее считалось, что для такого обучения необходимо пометать отдельные элементы последовательности (Gülçehre

and Bengio, 2013). Рекуррентные нейронные сети, в частности вышеупомянутая модель LSTM, теперь применяются для моделирования связей *последовательностей* с другими *последовательностями*, а не только с фиксированными входами. Такое обучение типа «последовательность в последовательность», похоже, привело к революции в другом приложении: машинном переводе (Sutskever et al., 2014; Bahdanau et al., 2015).

Эта тенденция к увеличению сложности была доведена до логического завершения с вводом в рассмотрение нейронных машин Тьюринга (Graves et al., 2014a), которые обучаются читать из ячеек памяти и писать произвольные данные в ячейки памяти. Такие нейронные сети могут обучаться простым программам на примерах желаемого поведения. Например, они могут научиться сортировать списки чисел, если им предъявить примеры отсортированных и неотсортированных последовательностей. Эта технология самопрограммирования пока находится в зачаточной стадии, но в будущем теоретически может быть применена почти к любой задаче.

Еще одним венцом глубокого обучения является обобщение на **обучение с подкреплением**. В этом контексте автономный агент должен научиться выполнять некоторое задание методом проб и ошибок, без какой-либо подсказки со стороны человека. Компания DeepMind продемонстрировала систему обучения с подкреплением, основанную на технологиях глубокого обучения, которая способна научиться играть в видеоигры Atari, достигая во многих случаях уровня, сравнимого с человеческим (Mnih et al., 2015). Глубокое обучение позволило также значительно усовершенствовать качество обучения с подкреплением в робототехнике (Finn et al., 2015).

Многие приложения глубокого обучения приносят солидную прибыль. Эти технологии используются в таких крупных компаниях, как Google, Microsoft, Facebook, IBM, Baidu, Apple, Adobe, Netflix, NVIDIA и NEC.

Прогресс в глубоком обучении сильно зависит от достижений в области программной инфраструктуры. Библиотеки Theano (Bergstra et al., 2010; Bastien et al., 2012), PyLearn2 (Goodfellow et al., 2013c), Torch (Collobert et al., 2011b), DistBelief (Dean et al., 2012), Caffe (Jia, 2013), MXNet (Chen et al., 2015) и TensorFlow (Abadi et al., 2015) применяются в важных исследовательских проектах и в коммерческих продуктах.

Глубокое обучение внесло вклад и в другие науки. Современные сверточные сети для распознавания объектов дают модель визуальной обработки, изучаемую в нейробиологии (DiCarlo, 2013). Кроме того, глубокое обучение предоставляет полезные инструменты для обработки больших массивов данных и прогнозирования в различных научных дисциплинах. Оно было успешно применено к прогнозированию взаимодействия молекул в интересах фармацевтических компаний, занимающихся поиском новых лекарств (Dahl et al., 2014), к поиску субатомных частиц (Baldi et al., 2014) и к автоматическому распознаванию сделанных микроскопом изображений для построения трехмерной карты человеческого мозга (Knowles-Barley et al., 2014). В будущем мы ожидаем проникновения глубокого обучения во все новые и новые отрасли науки.

Резюмируя, можно сказать, что глубокое обучение – это развивавшийся в течение нескольких десятилетий подход к машинному обучению, основанный главным образом на наших знаниях о человеческом мозге, на статистике и прикладной математике. В последние годы глубокое обучение переживает стремительный рост популярности и полезности благодаря в первую очередь появлению более мощных компьютеров, больших наборов данных и методов обучения более глубоких сетей. Будущее сулит нам новые проблемы и возможности дальнейшего совершенствования глубокого обучения с выходом на новые рубежи.

ОСНОВЫ ПРИКЛАДНОЙ МАТЕМАТИКИ И МАШИННОГО ОБУЧЕНИЯ

В этой части книги вводятся основные математические идеи, необходимые для понимания глубокого обучения. Мы начнем с общего математического аппарата: определения функций многих переменных, нахождения их минимумов и максимумов и численной оценки степени доверия.

Затем мы сформулируем фундаментальные цели машинного обучения и опишем, как их достичь с помощью задания модели, представляющей некоторые гипотезы, проектирования функции стоимости, которая измеряет степень соответствия гипотез реальности, и использования алгоритма обучения для минимизации функции стоимости.

Этот элементарный аппарат является основой для самых разнообразных алгоритмов машинного обучения, не обязательно глубокого. В последующих частях книги мы с его помощью разработаем алгоритмы глубокого обучения.

Линейная алгебра

Линейная алгебра – это раздел математики, который широко используется в науке и технике. Но поскольку линейная алгебра имеет дело с непрерывными, а не дискретными величинами, специалисты по информатике редко с ней сталкиваются. Однако для понимания многих алгоритмов машинного обучения, особенно глубокого, уверенное владение аппаратом линейной алгебры необходимо. Поэтому мы предположим введение в глубокое обучение концентрированное изложение основ линейной алгебры.

Если вы уже знакомы с линейной алгеброй, то можете пропустить эту главу. Если вам раньше доводилось встречаться с этим аппаратом, но нужен подробный справочник, чтобы освежить в памяти формулы, то рекомендуем книгу «The Matrix Cookbook» (Petersen and Pedersen, 2006). Если вы совсем ничего не знаете о линейной алгебре, то из этой главы вы почерпнете достаточно сведений для чтения книги, но мы настоятельно советуем обратиться к еще какому-нибудь источнику, посвященному исключительно линейной алгебре, например книге Шилова, Shilov (1977). В этой главе не рассматриваются многие вопросы линейной алгебры, не существенные для понимания глубокого обучения.

2.1. Скаляры, векторы, матрицы и тензоры

В линейной алгебре изучаются математические объекты нескольких видов.

- **Скаляры.** Скаляр – это просто число, в отличие от большинства других изучаемых в линейной алгебре объектов, представляющих собой массивы чисел. Мы будем обозначать скаляры курсивным шрифтом и, как правило, строчными буквами. Вводя в рассмотрение скаляр, мы будем указывать, что он означает, например: «Пусть $s \in \mathbb{R}$ обозначает угловой коэффициент прямой» (вещественное число) или «Обозначим $n \in \mathbb{N}$ число блоков» (целое натуральное число).
- **Векторы.** Вектор – это массив чисел. Элементы вектора расположены в определенном порядке. Отдельный элемент определяется своим индексом. Обычно векторы обозначаются строчными буквами курсивным полужирным шрифтом, например $\mathbf{x}$. Для обозначения элемента вектора указывается имя вектора курсивным шрифтом с подстрочным индексом. Первый элемент вектора $\mathbf{x}$ обозначается x_1 , второй – x_2 и т. д. Необходимо также указать тип хранящихся в векторе чисел. Если каждый элемент вектора принадлежит $\mathbb{R}$, а всего элементов n , то вектор принадлежит декартову произведению n экземпляров $\mathbb{R}$, оно обозначается $\mathbb{R}^n$. Если требуется явно перечислить элементы вектора, то они записываются в столбик в квадратных скобках:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}. \quad (2.1)$$

Можно считать, что векторам соответствуют точки в пространстве, а каждый элемент – координата вдоль одной из осей.

Иногда необходимо обозначить подмножество элементов вектора. В таком случае мы будем указывать в качестве подстрочного индекса множество индексов интересующих элементов. Например, чтобы адресовать элементы x_1 , x_3 и x_6 , мы определим множество $S = \{1, 3, 6\}$ и будем писать $\mathbf{x}_S$. Для обозначения дополнения ко множеству будем использовать знак $-$. Например, $\mathbf{x}_{-1}$ обозначает вектор, содержащий все элементы $\mathbf{x}$, кроме x_1 , а $\mathbf{x}_{-S}$ – вектор, содержащий все элементы $\mathbf{x}$, кроме x_1 , x_3 и x_6 .

- **Матрицы.** Матрица – это двумерный массив чисел, в котором каждый элемент идентифицируется двумя индексами, а не одним. Обычно матрицы обозначаются прописными буквами полужирным курсивным шрифтом, например $\mathbf{A}$. Если матрица вещественных чисел $\mathbf{A}$ имеет высоту m и ширину n , то говорят, что $\mathbf{A} \in \mathbb{R}^{m \times n}$. Элементы матрицы обычно обозначаются светлым курсивным шрифтом, а индексы перечисляются через запятую. Например, $A_{1,1}$ – элемент $\mathbf{A}$ в левом верхнем углу, а $A_{m,n}$ – в правом нижнем. Для адресации всех элементов в одной строке мы указываем номер этой строки i , а вместо номера столбца – двоеточие, например $A_{i,:}$ обозначает i -ю строку $\mathbf{A}$. Аналогично $A_{:,i}$ обозначает i -й столбец $\mathbf{A}$. Если требуется явно поименовать элементы матрицы, то мы записываем их в виде прямоугольной таблицы, заключенной в квадратные скобки:

$$\begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix}. \quad (2.2)$$

Иногда требуется индексировать матричные выражения, не обозначенные одной буквой. Тогда нижние индексы ставятся после всего выражения. Например, $f(\mathbf{A})_{i,j}$ обозначает элемент в позиции (i, j) матрицы, полученной применением функции f к матрице $\mathbf{A}$.

- **Тензоры.** Бывает, что нужен массив размерности, большей 2. В общем случае массив чисел в узлах равномерной сетки с произвольным числом осей называется тензором. Тензор с именем « $\mathbf{A}$ » будет обозначаться таким шрифтом: $\mathbf{A}$, а его элемент в позиции (i, j, k) – $A_{i,j,k}$.

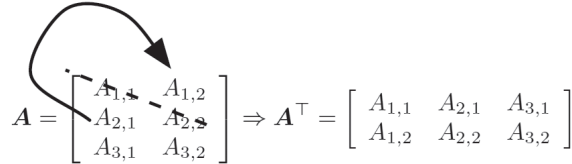
Важной операцией над матрицами является **транспонирование**. Транспонированной называется матрица, отраженная относительно главной диагонали, идущей из левого верхнего угла направо и вниз. На рис. 2.1 показано графическое изображение этой операции. Результат транспонирования матрицы $\mathbf{A}$ обозначается $\mathbf{A}^\top$ и формально определяется следующим образом:

$$(\mathbf{A}^\top)_{i,j} = A_{j,i}. \quad (2.3)$$

Векторы можно считать матрицами с одним столбцом. Тогда результатом транспонирования вектора будет матрица с одной строкой. Иногда мы будем записывать

вектор прямо в основном тексте в виде матрицы-строки, тогда транспонирование преобразует его в стандартный вектор-столбец, например: $\mathbf{x} = [x_1, x_2, x_3]^\top$.

Скаляр можно рассматривать как матрицу из одного элемента. Это значит, что результатом транспонирования скаляра является он сам: $a = a^\top$.



$$A = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \\ A_{3,1} & A_{3,2} \end{bmatrix} \Rightarrow A^\top = \begin{bmatrix} A_{1,1} & A_{2,1} & A_{3,1} \\ A_{1,2} & A_{2,2} & A_{3,2} \end{bmatrix}$$

Рис. 2.1 ❖ Транспонирование матрицы можно рассматривать как отражение относительно главной диагонали

Матрицы одинакового размера можно складывать поэлементно: $\mathbf{C} = \mathbf{A} + \mathbf{B}$ означает, что $C_{i,j} = A_{i,j} + B_{i,j}$.

Определены также операции сложения матрицы со скаляром и умножения матрицы на скаляр. Для этого нужно произвести соответствующую операцию для каждого элемента матрицы: $\mathbf{D} = a \cdot \mathbf{B} + c$ означает, что $D_{i,j} = a \cdot B_{i,j} + c$.

В контексте глубокого обучения применяется также не совсем традиционная нотация. Мы определяем операцию сложения матрицы и вектора, дающую матрицу: $\mathbf{C} = \mathbf{A} + \mathbf{b}$ означает, что $C_{i,j} = A_{i,j} + b_j$. Иными словами, вектор $\mathbf{b}$ прибавляется к каждой строке матрицы. Такая сокращенная запись позволяет обойтись без определения матрицы, в которой каждая строка содержит вектор $\mathbf{b}$. Это неявное копирование $\mathbf{b}$ сразу в несколько мест называется **укладыванием** (broadcasting).

2.2. Умножение матриц и векторов

Одна из самых важных операций над матрицами – перемножение двух матриц. Произведением матриц $\mathbf{A}$ и $\mathbf{B}$ также является матрица, $\mathbf{C}$. Она определена, только если число столбцов $\mathbf{A}$ равно числу строк $\mathbf{B}$. Если $\mathbf{A}$ имеет размер $m \times n$, а $\mathbf{B}$ – размер $n \times p$, то $\mathbf{C}$ будет иметь размер $m \times p$. Для обозначения произведения имена матриц записываются подряд, например:

$$\mathbf{C} = \mathbf{AB}. \quad (2.4)$$

Произведение определяется следующим образом:

$$C_{i,j} = \sum_k A_{i,k} B_{k,j}. \quad (2.5)$$

Отметим, что обычное произведение двух матриц получается не в результате **поэлементного перемножения**. Но такая операция тоже существует – она называется **произведением Адамара** и обозначается $\mathbf{A} \odot \mathbf{B}$.

Скалярным произведением двух векторов $\mathbf{x}$ и $\mathbf{y}$ одинаковой размерности называется произведение матриц $\mathbf{x}^\top \mathbf{y}$. Таким образом, элемент $C_{i,j}$ матрицы $\mathbf{C} = \mathbf{AB}$ является скалярным произведением i -й строки $\mathbf{A}$ и j -го столбца $\mathbf{B}$.

Операции над матрицами обладают рядом полезных свойств, упрощающих их математический анализ. Например, операция умножения дистрибутивна относительно сложения:

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}. \quad (2.6)$$

Она также ассоциативна:

$$\mathbf{A}(\mathbf{BC}) = (\mathbf{AB})\mathbf{C}. \quad (2.7)$$

В отличие от умножения скаляров, умножение матриц не коммутативно (равенство $\mathbf{AB} = \mathbf{BA}$ справедливо не всегда). Но скалярное произведение векторов коммутативно:

$$\mathbf{x}^\top \mathbf{y} = \mathbf{y}^\top \mathbf{x}. \quad (2.8)$$

Операции транспонирования и умножения связаны простой формулой:

$$(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top. \quad (2.9)$$

Отсюда следует доказательство тождества (2.8), поскольку скалярное произведение – это скаляр и, значит, совпадает с результатом транспонирования:

$$\mathbf{x}^\top \mathbf{y} = (\mathbf{x}^\top \mathbf{y})^\top = \mathbf{y}^\top \mathbf{x}. \quad (2.10)$$

Поскольку эта книга – не учебник по линейной алгебре, мы не станем приводить полный список полезных свойств матричного произведения, а просто сообщим, что их еще очень много.

Теперь мы знаем о принятых в линейной алгебре обозначениях достаточно, для того чтобы записать систему линейных уравнений:

$$\mathbf{Ax} = \mathbf{b}, \quad (2.11)$$

где $\mathbf{A} \in \mathbb{R}^{m \times n}$ – известная матрица, $\mathbf{b} \in \mathbb{R}^m$ – известный вектор, а $\mathbf{x} \in \mathbb{R}^n$ – неизвестный вектор, элементы которого, x_i , мы хотим найти. Каждая строка $\mathbf{A}$ и соответственный элемент $\mathbf{b}$ дают одно ограничение. Уравнение (2.11) можно переписать в виде:

$$\mathbf{A}_{1,\cdot} \mathbf{x} = b_1 \quad (2.12)$$

$$\mathbf{A}_{2,\cdot} \mathbf{x} = b_2 \quad (2.13)$$

$$\dots \quad (2.14)$$

$$\mathbf{A}_{m,\cdot} \mathbf{x} = b_m \quad (2.15)$$

или даже в еще более явном виде:

$$\mathbf{A}_{1,1}x_1 + \mathbf{A}_{1,2}x_2 + \dots + \mathbf{A}_{1,n}x_n = b_1 \quad (2.16)$$

$$\mathbf{A}_{2,1}x_1 + \mathbf{A}_{2,2}x_2 + \dots + \mathbf{A}_{2,n}x_n = b_2 \quad (2.17)$$

$$\dots \quad (2.18)$$

$$\mathbf{A}_{m,1}x_1 + \mathbf{A}_{m,2}x_2 + \dots + \mathbf{A}_{m,n}x_n = b_m \quad (2.19)$$

Нотация умножения матрицы на вектор позволяет записывать такие уравнения более компактно.

2.3. Единичная и обратная матрица

В линейной алгебре определена операция обращения матриц, которая позволяет аналитически решить уравнение (2.11) для многих значений $\mathbf{A}$.

Чтобы описать операцию обращения, мы должны сначала определить **единичную матрицу**. Так называется матрица, которая при умножении на любой вектор оставля-

ет этот вектор без изменения. Единичную матрицу, сохраняющую n -мерные векторы, будем обозначать I_n . Формально говоря, $I_n \in \mathbb{R}^{n \times n}$ и

$$\forall \mathbf{x} \in \mathbb{R}^n, I_n \mathbf{x} = \mathbf{x}. \quad (2.20)$$

Единичная матрица устроена просто: все элементы главной диагонали равны единице, а все остальные – нулю. Пример приведен на рис. 2.2.

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Рис. 2.2 ❖ Пример единичной матрицы I_3

Матрица, обратная к A , обозначается A^{-1} и по определению удовлетворяет следующему соотношению:

$$A^{-1}A = I_n. \quad (2.21)$$

Теперь для решения уравнения (2.11) нужно проделать следующие действия:

$$A\mathbf{x} = \mathbf{b}; \quad (2.22)$$

$$A^{-1}A\mathbf{x} = A^{-1}\mathbf{b}; \quad (2.23)$$

$$I_n\mathbf{x} = A^{-1}\mathbf{b}; \quad (2.24)$$

$$\mathbf{x} = A^{-1}\mathbf{b}. \quad (2.25)$$

Разумеется, эта процедура предполагает, что матрицу A^{-1} можно найти. В следующем разделе мы обсудим условия существования обратной матрицы.

В случае, когда A^{-1} существует, для ее нахождения в замкнутой форме можно применить один из нескольких алгоритмов. Теоретически один раз найденную обратную матрицу можно использовать многократно для решения уравнения с разными значениями $\mathbf{b}$. Однако на практике A^{-1} редко используется в программах. Поскольку матрицу A^{-1} можно представить в компьютере лишь с ограниченной точностью, алгоритмы, в которых используется значение $\mathbf{b}$, обычно дают более точные оценки $\mathbf{x}$.

2.4. Линейная зависимость и линейная оболочка

Для существования A^{-1} уравнение (2.11) должно иметь единственное решение для любого значения $\mathbf{b}$. Бывает и так, что система уравнений не имеет ни одного решения или имеет бесконечно много решений для некоторых значений $\mathbf{b}$. Но никогда не может быть так, что система имеет конечное число решений, большее 1; если $\mathbf{x}$ и $\mathbf{y}$ – решения, то

$$\mathbf{z} = \alpha\mathbf{x} + (1 - \alpha)\mathbf{y} \quad (2.26)$$

тоже решение при любом вещественном α .

Чтобы проанализировать, сколько решений имеет уравнение, представим себе, что столбцы A определяют различные направления от **начала координат** (точки, соответствующей вектору, все элементы которого равны нулю), а затем подумаем, сколько

есть способов достичь точки $\mathbf{b}$. Тогда элемент x_i определяет, как далеко следует пройти в направлении столбца i :

$$\mathbf{Ax} = \sum_i x_i \mathbf{A}_{:,i}. \quad (2.27)$$

В общем случае такое выражение называется **линейной комбинацией**. Формально для получения линейной комбинации некоторого множества векторов $\{\mathbf{v}^{(1)}, \dots, \mathbf{v}^{(n)}\}$ нужно умножить каждый вектор $\mathbf{v}^{(i)}$ на скалярный коэффициент и сложить результаты:

$$\sum_i c_i \mathbf{v}^{(i)}. \quad (2.28)$$

Линейной оболочкой множества векторов называется множество всех векторов, представимых в виде их линейных комбинаций.

Таким образом, чтобы определить, имеет ли уравнение $\mathbf{Ax} = \mathbf{b}$ решение, нужно проверить, входит ли $\mathbf{b}$ в линейную оболочку столбцов матрицы $\mathbf{A}$. Эта линейная оболочка называется **пространством столбцов**, или **областью значений** матрицы $\mathbf{A}$.

Итак, чтобы система $\mathbf{Ax} = \mathbf{b}$ имела решение для любого $\mathbf{b} \in \mathbb{R}^m$, необходимо, чтобы пространство столбцов $\mathbf{A}$ совпадало с $\mathbb{R}^m$. Если в $\mathbb{R}^m$ существует точка $\mathbf{b}$, не принадлежащая пространству столбцов, то для такого значения $\mathbf{b}$ система не будет иметь решения. Из требования о совпадении пространства столбцов $\mathbf{A}$ с $\mathbb{R}^m$ сразу следует, что в $\mathbf{A}$ должно быть по меньшей мере m столбцов, т. е. $n \geq m$. Иначе размерность пространства столбцов была бы меньше m . Рассмотрим, к примеру, матрицу 3×2 . Правая часть $\mathbf{b}$ имеет размерность 3, а размерность $\mathbf{x}$ равна всего 2, поэтому путем изменения $\mathbf{x}$ мы в лучшем случае сможем заместить двумерную плоскость в $\mathbb{R}^3$. Уравнение будет иметь решение тогда и только тогда, когда $\mathbf{b}$ лежит в этой плоскости.

Условие $n \geq m$ необходимо, чтобы система имела решение для любой правой части, но недостаточно, поскольку некоторые столбцы могут быть избыточны. Рассмотрим матрицу 2×2 , в которой оба столбца одинаковы. Ее пространство столбцов такое же, как у матрицы 2×1 , содержащей только один экземпляр повторяющегося столбца. Иными словами, пространство столбцов – прямая, не покрывающая все $\mathbb{R}^2$, несмотря на то что столбцов два.

Такой вид избыточности называется **линейной зависимостью**. Множество векторов называется **линейно независимым**, если ни один вектор из этого множества не является линейной комбинацией других. Если добавить во множество вектор, являющийся линейной комбинацией каких-то других, то линейная оболочка исходного множества не пополнится новыми точками. Следовательно, для того чтобы пространство столбцов матрицы совпадало со всем пространством $\mathbb{R}^m$, матрица должна содержать, по меньшей мере, один набор m линейно независимых столбцов. Это необходимое и достаточное условие существования у уравнения (2.11) решения для любого значения $\mathbf{b}$. Отметим, что в условии говорится о наличии в точности m линейно независимых столбцов в наборе, а не хотя бы m . Ни один набор m -мерных векторов не может содержать более m линейно независимых векторов, но матрица, имеющая более m столбцов, может содержать несколько таких наборов.

Для существования обратной матрицы необходимо еще, чтобы уравнение (2.11) имело не более одного решения для каждого значения $\mathbf{b}$. Это означает, что в матрице может быть не больше m столбцов, иначе существовало бы более одного способа параметризовать каждое решение.

Собирая все вместе, мы заключаем, что матрица должна быть квадратной, т. е. $m = n$, и что ее столбцы должны быть линейно независимы. Квадратная матрица с линейно зависимыми столбцами называется **вырожденной** (особенной, сингулярной).

Если A не квадратная или квадратная, но вырожденная, то решить уравнение все-таки можно, но метод обращения матрицы не подойдет.

До сих пор при обсуждении обратных матриц мы имели в виду умножение слева. Но можно также определить матрицу, обратную относительно умножения справа:

$$AA^{-1} = I. \quad (2.29)$$

Для квадратных матриц обе обратные матрицы совпадают.

2.5. Нормы

Иногда требуется получить длину вектора. В машинном обучении для измерения длины применяются функции, называемые **нормами**. Норма L^p определяется следующим образом:

$$\|\mathbf{x}\|_p = \left(\sum_i |x_i|^p \right)^{\frac{1}{p}}. \quad (2.30)$$

для $p \in \mathbb{R}, p \geq 1$.

Нормы, в т. ч. и норма L^p , – это функции, отображающие векторы на неотрицательные числа. Интуитивно норма вектора $\mathbf{x}$ измеряет расстояние от точки $\mathbf{x}$ до начала координат. Более строго, нормой называется любая функция f , обладающая следующими свойствами:

- $f(\mathbf{x}) = 0 \Rightarrow \mathbf{x} = \mathbf{0}$;
- $f(\mathbf{x} + \mathbf{y}) \leq f(\mathbf{x}) + f(\mathbf{y})$ (неравенство треугольника);
- $\forall \alpha \in \mathbb{R} f(\alpha \mathbf{x}) = |\alpha| f(\mathbf{x})$.

Норма L^2 называется **евклидовой нормой**, это просто евклидово расстояние от начала координат до точки, определяемой вектором $\mathbf{x}$. Норма L^2 применяется в машинном обучении так часто, что ее обозначают просто $\|\mathbf{x}\|$, опуская надстрочный индекс 2. Кроме того, длину вектора принято измерять как квадрат нормы L^2 , который можно записать в виде $\mathbf{x}^\top \mathbf{x}$.

Квадрат нормы L^2 удобнее самой нормы с точки зрения математических выкладок и вычислений. Например, частная производная квадрата нормы L^2 по каждому элементу $\mathbf{x}$ зависит только от этого элемента, тогда как производные самой нормы L^2 зависят от всего вектора в целом. Во многих контекстах квадрат нормы L^2 нежелателен, потому что он слишком медленно растет вблизи начала координат. В некоторых приложениях машинного обучения важно различать элементы, в точности равные нулю и мало отличающиеся от нуля. В таких случаях мы прибегаем к функции, которая всюду растет с одинаковой скоростью, но сохраняет математическую простоту: норме L^1 . Норму L^1 можно записать в виде:

$$\|\mathbf{x}\|_1 = \sum_i |x_i|. \quad (2.31)$$

Эта норма часто используется в машинном обучении, когда важно различать нулевые и ненулевые элементы. Всякий раз как элемент вектора $\mathbf{x}$ отдалается от 0 на ε , норма L^1 увеличивается на ε .

Иногда длина вектора измеряется количеством его ненулевых элементов. Некоторые авторы называют такую функцию «нормой L^0 », но такая терминология некорректна. Количество ненулевых элементов вектора нормой не является, поскольку при умножении вектора на α это количество не меняется. Зачастую вместо подсчета ненулевых элементов используют норму L^1 .

В машинном обучении также часто возникает норма L^∞ , которую еще называют **максимальной нормой**. Она определяется как максимальное абсолютное значение элементов вектора:

$$\|\mathbf{x}\|_\infty = \max_i |x_i|. \quad (2.32)$$

Иногда возникает необходимость оценить размер матрицы. В контексте глубокого обучения для этого обычно применяют **норму Фробениуса**:

$$\|A\|_F = \sqrt{\sum_{i,j} A_{i,j}^2}, \quad (2.33)$$

аналогичную норме L^2 вектора.

Скалярное произведение двух векторов можно выразить через нормы:

$$\mathbf{x}^\top \mathbf{y} = \|\mathbf{x}\|_2 \|\mathbf{y}\|_2 \cos \theta, \quad (2.34)$$

где θ – угол между $\mathbf{x}$ и $\mathbf{y}$.

2.6. Специальные виды матриц и векторов

Некоторые частные случаи матриц и векторов особенно полезны.

В **диагональной матрице** все элементы, кроме находящихся на главной диагонали, равны нулю. Точнее, матрица $\mathbf{D}$ называется диагональной, если $D_{i,j} = 0$ для всех $i \neq j$. Примером диагональной матрицы служит единичная матрица, в которой все элементы на главной диагонали равны 1. Квадратная диагональная матрица, диагональ которой описывается вектором $\mathbf{v}$, обозначается $\text{diag}(\mathbf{v})$. Диагональные матрицы представляют особый интерес, потому что произведение с такой матрицей вычисляется очень эффективно. Чтобы вычислить $\text{diag}(\mathbf{v})\mathbf{x}$, нужно умножить каждый элемент x_i на v_i . Иначе говоря, $\text{diag}(\mathbf{v})\mathbf{x} = \mathbf{v} \odot \mathbf{x}$. Обратить квадратную диагональную матрицу тоже просто. Обратная к ней матрица существует, только если все диагональные элементы отличны от нуля, и тогда $\text{diag}(\mathbf{v})^{-1} = \text{diag}([1/v_1, \dots, 1/v_n]^\top)$. Во многих случаях алгоритм машинного обучения можно сформулировать в терминах произвольных матриц, но получить менее накладный (но и менее общий) алгоритм, ограничившись только диагональными матрицами.

Диагональная матрица может быть не только квадратной, но и прямоугольной. У неквадратной диагональной матрицы нет обратной, но умножение на нее все равно обходится дешево. Произведение диагональной матрицы $\mathbf{D}$ на вектор $\mathbf{x}$ сводится к умножению каждого элемента $\mathbf{x}$ на некоторое число и дописыванию нулей в конец получившегося вектора, если высота $\mathbf{D}$ больше ее ширины, или отбрасыванию последних элементов вектора в противном случае.

Матрица называется **симметричной**, если совпадает с транспонированной:

$$\mathbf{A} = \mathbf{A}^\top. \quad (2.35)$$

Симметричные матрицы часто образуются, когда их элементы порождаются некоторой функцией двух аргументов, результат которой не зависит от порядка аргументов. Например, если $A_{i,j}$ – расстояние от точки i до точки j , то $A_{i,j} = A_{j,i}$, поскольку функция расстояния симметрична.

Единичным вектором называется вектор с нормой, равной 1:

$$\|\mathbf{x}\|_2 = 1. \quad (2.36)$$

Говорят, что векторы $\mathbf{x}$ и $\mathbf{y}$ ортогональны, если $\mathbf{x}^\top \mathbf{y} = 0$. Если нормы обоих векторов не равны 0, то это значит, что угол между векторами составляет 90 градусов. В пространстве $\mathbb{R}^n$ можно найти не более n взаимно ортогональных ненулевых векторов. Ортогональные векторы, норма которых равна 1, называются **ортонормированными**.

Ортогональной называется квадратная матрица, строки и столбцы которой образуют ортонормированные системы векторов:

$$\mathbf{A}^\top \mathbf{A} = \mathbf{A} \mathbf{A}^\top = \mathbf{I}. \quad (2.37)$$

Отсюда следует, что

$$\mathbf{A}^{-1} = \mathbf{A}^\top. \quad (2.38)$$

Ортогональные матрицы интересны из-за простоты вычисления обратной матрицы. Обратите внимание на определение ортогональной матрицы: ее строки должны образовывать не просто ортогональную, а ортонормированную систему векторов. Не существует специального термина для матриц, строки и столбцы которых образуют ортогональную, но не ортонормированную систему векторов.

2.7. Спектральное разложение матрицы

Многие математические объекты удастся лучше понять, если разложить их на составные части или найти какие-то универсальные свойства, не зависящие от способа представления.

Например, целые числа можно разложить на простые множители. Способ представления числа 12 зависит от того, в какой системе счисления оно записано, например двоичной или десятичной, но в любом случае $12 = 2 \times 2 \times 3$. Из этого представления можно вывести ряд полезных свойств, например что 12 не делится на 5 или что любое число, делящееся на 12, делится также на 3.

Таким образом, истинная природа целого числа раскрывается в результате разложения его на простые множители. И точно так же мы можем разложить матрицу и тем самым получить о ее функциональных свойствах некоторую информацию, не очевидную из представления матрицы в виде массива элементов.

Одно из самых популярных разложений матрицы называется **спектральным разложением**, или разложением на множество собственных векторов и собственных значений.

Собственным вектором квадратной матрицы $\mathbf{A}$ называется ненулевой вектор $\mathbf{v}$ – такой, что умножение $\mathbf{A}$ на $\mathbf{v}$ изменяет лишь масштаб $\mathbf{v}$:

$$\mathbf{A} \mathbf{v} = \lambda \mathbf{v}. \quad (2.39)$$

Скаляр λ называется **собственным значением**, соответствующим этому собственному вектору. (Можно также искать левые собственные векторы, для которых $\mathbf{v}^T \mathbf{A} = \lambda \mathbf{v}^T$, но обычно нас интересуют только правые собственные векторы.)

Если $\mathbf{v}$ – собственный вектор $\mathbf{A}$, то собственным будет и вектор $s\mathbf{v}$ для любого $s \in \mathbb{R}$, $s \neq 0$. Более того, вектору $s\mathbf{v}$ соответствует то же собственное значение, что и $\mathbf{v}$. Поэтому мы обычно ищем только единичные собственные векторы.

Пусть матрица $\mathbf{A}$ имеет n линейно независимых собственных векторов $\{\mathbf{v}^{(1)}, \dots, \mathbf{v}^{(n)}\}$ с собственными значениями $\{\lambda_1, \dots, \lambda_n\}$. Образует из них матрицу $\mathbf{V}$, в которой каждый столбец – это собственный вектор: $\mathbf{V} = [\mathbf{v}^{(1)}, \dots, \mathbf{v}^{(n)}]$. А из собственных значений образуем вектор $\boldsymbol{\lambda} = [\lambda_1, \dots, \lambda_n]^T$. Тогда **спектральное разложение** матрицы $\mathbf{A}$ описывается формулой:

$$\mathbf{A} = \mathbf{V} \text{diag}(\boldsymbol{\lambda}) \mathbf{V}^{-1}. \quad (2.40)$$

Мы видели, что конструирование матриц с заданными собственными значениями и собственными векторами позволяет растягивать пространство в нужных направлениях. Но часто бывает нужно разложить имеющуюся матрицу по ее собственным векторам и собственным значениям. Это помогает анализировать некоторые свойства матрицы точно так же, как разложение целого числа на простые множители помогает понять поведение этого числа.

Не у каждой матрицы есть спектральное разложение. Иногда спектральное разложение существует, но состоит из комплексных, а не вещественных чисел. К счастью, в этой книге нам обычно придется иметь дело только с матрицами специального вида, у которых имеется простое разложение. Точнее, у любой симметричной вещественной матрицы все собственные векторы и собственные значения вещественные.

$$\mathbf{A} = \mathbf{Q} \boldsymbol{\Lambda} \mathbf{Q}^T, \quad (2.41)$$

где $\mathbf{Q}$ – ортогональная матрица, образованная собственными векторами $\mathbf{A}$, а $\boldsymbol{\Lambda}$ – диагональная матрица. Собственное значение $\boldsymbol{\Lambda}_{i,i}$ ассоциировано с собственным вектором в i -м столбце $\mathbf{Q}$, обозначаемым $\mathbf{Q}_{:,i}$. Поскольку $\mathbf{Q}$ – ортогональная матрица, можно считать, что $\mathbf{A}$ масштабирует пространство с коэффициентом λ_i в направлении $\mathbf{v}^{(i)}$. На рис. 2.3 показан пример.

Хотя для любой симметричной вещественной матрицы $\mathbf{A}$ существует спектральное разложение, это разложение может быть не единственным. Если какие-то два или более собственных векторов имеют одинаковое собственное значение, то любые ортогональные векторы, принадлежащие их линейной оболочке, также будут собственными векторами $\mathbf{A}$ с тем же самым собственным значением, поэтому матрицу $\mathbf{Q}$ можно с тем же успехом образовать из этих векторов. По принятому соглашению элементы $\boldsymbol{\Lambda}$ обычно располагаются в порядке убывания. При таком соглашении спектральное разложение единственно, только если все собственные значения различны.

Спектральное разложение может многое рассказать о матрице. Матрица является вырожденной тогда и только тогда, когда у нее есть хотя бы одно нулевое собственное значение. Кроме того, спектральное разложение вещественной симметричной матрицы можно использовать для оптимизации квадратичных форм вида $f(\mathbf{x}) = \mathbf{x}^T \mathbf{A} \mathbf{x}$ при условии $\|\mathbf{x}\|_2 = 1$. Если $\mathbf{x}$ – собственный вектор $\mathbf{A}$, то f равно его собственному значению. Максимальное (минимальное) значение f при заданном ограничении равно максимальному (минимальному) собственному значению.

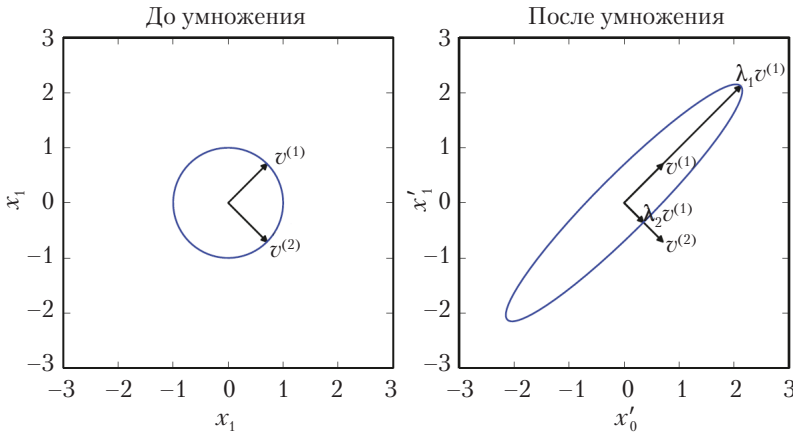


Рис. 2.3 ❖ Геометрический смысл собственных векторов и собственных значений. У матрицы $\mathbf{A}$ два ортонормированных собственных вектора: $\mathbf{v}^{(1)}$ с собственным значением λ_1 и $\mathbf{v}^{(2)}$ с собственным значением λ_2 . (Слева) Множество всех единичных векторов $\mathbf{u} \in \mathbb{R}^2$ изображено в виде единичной окружности. (Справа) Изображено множество точек $\mathbf{A}\mathbf{u}$. Наблюдая, во что $\mathbf{A}$ переводит единичную окружность, мы можем сделать вывод, что она масштабирует пространство в направлении $\mathbf{v}^{(i)}$ с коэффициентом λ_i

Матрица, все собственные значения которой положительны, называется **положительно определенной**. Если же все собственные значения положительны или равны нулю, то матрица называется **положительно полуопределенной**. Аналогично, если все собственные значения отрицательны, то матрица называется **отрицательно определенной**, а если отрицательны или равны нулю – то **отрицательно полуопределенной**. Положительно полуопределенные матрицы интересны тем, что для них выполняется неравенство $\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$ при любом $\mathbf{x}$. Положительная определенность дополнительно гарантирует, что $\mathbf{x}^\top \mathbf{A} \mathbf{x} = 0 \Rightarrow \mathbf{x} = 0$.

2.8. Сингулярное разложение

В разделе 2.7 мы видели, как разложить матрицу по собственным векторам и собственным значениям. **Сингулярное разложение** (singular value decomposition – SVD) – это другой способ разложения матрицы: по **сингулярным векторам** и **сингулярным значениям**. SVD несет ту же информацию, что спектральное разложение, но применимо в более общем случае. Сингулярное разложение есть у любой вещественной матрицы, чего не скажешь о спектральном разложении. Например, если матрица не квадратная, то спектральное разложение не определено, поэтому мы вынуждены использовать сингулярное разложение.

Напомним, что для получения спектрального разложения матрицы $\mathbf{A}$ нужно найти матрицу $\mathbf{V}$ собственных векторов и вектор $\boldsymbol{\lambda}$ собственных значений – такие, что:

$$\mathbf{A} = \mathbf{V} \text{diag}(\boldsymbol{\lambda}) \mathbf{V}^{-1}. \quad (2.42)$$

Сингулярное разложение аналогично, но теперь $\mathbf{A}$ записывается в виде произведения трех матриц:

$$\mathbf{A} = \mathbf{UDV}^T. \quad (2.43)$$

Пусть $\mathbf{A}$ – матрица $m \times n$. Тогда $\mathbf{U}$ должна быть матрицей $m \times m$, $\mathbf{D}$ – матрицей $m \times n$, а $\mathbf{V}$ – матрицей $n \times n$.

Эти матрицы обладают определенными свойствами. Матрицы $\mathbf{U}$ и $\mathbf{V}$ ортогональные, а матрица $\mathbf{D}$ диагональная (при этом необязательно квадратная).

Элементы на диагонали $\mathbf{D}$ называются **сингулярными значениями** матрицы $\mathbf{A}$. Столбцы $\mathbf{U}$ называются **левыми сингулярными векторами**, а столбцы $\mathbf{V}$ – **правыми сингулярными векторами**.

Сингулярное разложение $\mathbf{A}$ можно интерпретировать в терминах спектрального разложения функций $\mathbf{A}$. Левые сингулярные векторы $\mathbf{A}$ являются собственными векторами $\mathbf{AA}^T$, а правые сингулярные векторы $\mathbf{A}$ – собственными векторами $\mathbf{A}^T\mathbf{A}$. Ненулевые сингулярные значения $\mathbf{A}$ равны квадратным корням из собственных значений $\mathbf{A}^T\mathbf{A}$ (и $\mathbf{AA}^T$).

Но, пожалуй, самое полезное свойство сингулярного разложения – использование его для обобщения операции обращения матриц на неквадратные матрицы, о чем мы и поговорим в следующем разделе.

2.9. Псевдообратная матрица Мура–Пенроуза

Операция обращения определена только для квадратных матриц. Допустим, что требуется найти левую обратную матрицу $\mathbf{B}$ для матрицы $\mathbf{A}$, что позволило бы решить линейное уравнение

$$\mathbf{Ax} = \mathbf{y} \quad (2.44)$$

путем умножения обеих частей слева на $\mathbf{B}$:

$$\mathbf{x} = \mathbf{By} \quad (2.45)$$

Единственное отображение $\mathbf{A}$ на $\mathbf{B}$ возможно не всегда; это зависит от характера задачи.

Если высота $\mathbf{A}$ больше ширины, то у такого уравнения может не оказаться решений. Если же ширина $\mathbf{A}$ больше высоты, то решений может быть много.

Операция **псевдообращения Мура–Пенроуза** позволяет кое-что сделать и в этих случаях. Псевдообратная к $\mathbf{A}$ матрица определяется следующим образом:

$$\mathbf{A}^+ = \lim_{\alpha \searrow 0} (\mathbf{A}^T\mathbf{A} + \alpha\mathbf{I})^{-1}\mathbf{A}^T. \quad (2.46)$$

Применяемые на практике алгоритмы вычисления псевдообратной матрицы основаны не на этом определении, а на формуле

$$\mathbf{A}^+ = \mathbf{VD}^+\mathbf{U}^T, \quad (2.47)$$

где $\mathbf{U}$, $\mathbf{D}$ и $\mathbf{V}$ составляют сингулярное разложение $\mathbf{A}$, а псевдообратная матрица $\mathbf{D}^+$ диагональной матрицы $\mathbf{D}$ получается путем обращения всех ненулевых диагональных элементов с последующим транспонированием.

Если число столбцов $\mathbf{A}$ больше числа строк, то решение линейного уравнения, найденное псевдообращением, – лишь одно из многих возможных. Точнее, будет найдено решение $\mathbf{x} = \mathbf{A}^+\mathbf{y}$ с минимальной евклидовой нормой $\|\mathbf{x}\|_2$.

Если число строк $\mathbf{A}$ больше числа столбцов, то может не оказаться ни одного решения. В таком случае псевдообращение находит такой вектор $\mathbf{x}$, для которого $\mathbf{Ax}$ максимально близко к $\mathbf{y}$ в терминах евклидовой нормы $\|\mathbf{Ax} - \mathbf{y}\|_2$.

2.10. Оператор следа

Оператор следа вычисляет сумму всех диагональных элементов матрицы:

$$\text{Tr}(\mathbf{A}) = \sum_i \mathbf{A}_{i,i}. \quad (2.48)$$

Этот оператор полезен по многим причинам. Некоторые операции, которые трудно выразить, не прибегая к нотации суммирования, можно записать с помощью произведения матриц и оператора следа. Вот, например, как можно переписать определение нормы Фробениуса матрицы:

$$\|\mathbf{A}\|_F = \sqrt{\text{Tr}(\mathbf{AA}^\top)}. \quad (2.49)$$

Если в выражение входит оператор следа, то открываются возможности преобразовывать это выражение, пользуясь различными тождествами. Например, след инвариантен относительно транспонирования:

$$\text{Tr}(\mathbf{A}) = \text{Tr}(\mathbf{A}^\top). \quad (2.50)$$

След квадратной матрицы, представленной в виде произведения сомножителей, инвариантен также относительно перемещения последнего сомножителя в начало, если формы матриц допускают такую операцию:

$$\text{Tr}(\mathbf{ABC}) = \text{Tr}(\mathbf{CAB}) = \text{Tr}(\mathbf{BCA}), \quad (2.51)$$

или в более общем виде:

$$\text{Tr}\left(\prod_{i=1}^n \mathbf{F}^{(i)}\right) = \text{Tr}\left(\mathbf{F}^{(n)} \prod_{i=1}^{n-1} \mathbf{F}^{(i)}\right). \quad (2.52)$$

Эта инвариантность относительно циклической перестановки имеет место даже тогда, когда меняется форма произведения. Например, если $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{B} \in \mathbb{R}^{n \times m}$, то

$$\text{Tr}(\mathbf{AB}) = \text{Tr}(\mathbf{BA}), \quad (2.53)$$

несмотря на то что $\mathbf{AB} \in \mathbb{R}^{m \times m}$, а $\mathbf{BA} \in \mathbb{R}^{n \times n}$.

Полезно также помнить, что след скаляра равен ему самому: $a = \text{Tr}(a)$.

2.11. Определитель

Определитель квадратной матрицы, обозначаемый $\det(\mathbf{A})$, – это функция, сопоставляющая матрице вещественное число. Определитель равен произведению всех собственных значений матрицы. Абсолютную величину определителя можно рассматривать как меру сжатия или расширения пространства матрицей. Если определитель равен 0, то пространство полностью сворачивается хотя бы по одному измерению, т. е. теряется весь объем. Если определитель равен 1, то преобразование сохраняет объем.

2.12. Пример: метод главных компонент

Один из простых алгоритмов машинного обучения, метод главных компонент (principal components analysis – PCA), можно вывести из основ линейной алгебры.

Пусть имеется набор m точек $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ в пространстве $\mathbb{R}^n$, и мы хотим подвергнуть их сжатию с потерей информации, т. е. сохранить точки в меньшем объеме памяти, возможно, ценой некоторой потери точности. Но хотелось бы свести эти потери к минимуму.

Один из способов кодирования точек – представить их в пространстве меньшей размерности. Для каждой точки $\mathbf{x}^{(i)} \in \mathbb{R}^n$ мы ищем соответствующий ей кодовый вектор $\mathbf{c}^{(i)} \in \mathbb{R}^l$. Если l меньше n , то для хранения кодированных точек потребуется меньше памяти, чем для исходных. Мы хотим найти функцию кодирования $f(\mathbf{x}) = \mathbf{c}$ и функцию декодирования, реконструирующую исходную точку по кодированной, $\mathbf{x} \approx g(f(\mathbf{x}))$.

Алгоритм PCA определяется выбором функции декодирования. Чтобы упростить декодер, мы будем использовать умножение матриц для обратного перехода в $\mathbb{R}^n$. Пусть $g(\mathbf{c}) = \mathbf{D}\mathbf{c}$, где $\mathbf{D} \in \mathbb{R}^{n \times l}$ – матрица, определяющая декодирование.

Вычисление оптимального декодера может оказаться трудной задачей. Для ее упрощения в методе PCA на матрицу $\mathbf{D}$ налагается ограничение: ее столбцы должны быть ортогональны (но это не означает, что $\mathbf{D}$ – ортогональная матрица в определенном выше смысле, разве что $l = n$).

У поставленной задачи бесконечно много решений, поскольку мы можем увеличить масштаб $D_{:,p}$ одновременно пропорционально уменьшив c_i для всех точек. Для получения единственного решения потребуем еще, чтобы норма всех столбцов $\mathbf{D}$ была равна 1.

Чтобы превратить смутную идею в алгоритм, нужно первым делом решить, как генерировать оптимальную кодовую точку $\mathbf{c}^*$ для каждой исходной точки $\mathbf{x}$. Один из способов – минимизировать расстояние между $\mathbf{x}$ и ее реконструкцией $g(\mathbf{c}^*)$. Для измерения этого расстояния применим какую-нибудь норму. В алгоритме главных компонент берется норма L^2 :

$$\mathbf{c}^* = \arg \min_{\mathbf{c}} \|\mathbf{x} - g(\mathbf{c})\|_2. \quad (2.54)$$

Мы можем использовать квадрат нормы L^2 , а не ее саму, потому что минимум достигается при одном и том же значении $\mathbf{c}$, т. к. норма L^2 неотрицательна, а операция возведения в квадрат – монотонно возрастающая для неотрицательных аргументов.

$$\mathbf{c}^* = \arg \min_{\mathbf{c}} \|\mathbf{x} - g(\mathbf{c})\|_2^2. \quad (2.55)$$

Минимизируемую функцию можно переписать в виде:

$$(\mathbf{x} - g(\mathbf{c}))^\top (\mathbf{x} - g(\mathbf{c})) \quad (2.56)$$

(по определению нормы L^2 из формулы (2.30))

$$= \mathbf{x}^\top \mathbf{x} - \mathbf{x}^\top g(\mathbf{c}) - g(\mathbf{c})^\top \mathbf{x} + g(\mathbf{c})^\top g(\mathbf{c}) \quad (2.57)$$

(в силу дистрибутивности)

$$= \mathbf{x}^\top \mathbf{x} - 2\mathbf{x}^\top g(\mathbf{c}) + g(\mathbf{c})^\top g(\mathbf{c}) \quad (2.58)$$

(поскольку результат транспонирования скаляра $g(\mathbf{c})^\top \mathbf{x}$ совпадает с ним самим).

Снова изменим минимизируемую функцию, опустив первый член, не зависящий от $\mathbf{c}$:

$$\mathbf{c}^* = \arg \min_{\mathbf{c}} -2\mathbf{x}^\top g(\mathbf{c}) + g(\mathbf{c})^\top g(\mathbf{c}). \quad (2.59)$$

Теперь подставим сюда определение $g(\mathbf{c})$:

$$\mathbf{c}^* = \arg \min_{\mathbf{c}} -2\mathbf{x}^\top \mathbf{D}\mathbf{c} + \mathbf{c}^\top \mathbf{D}^\top \mathbf{D}\mathbf{c} \quad (2.60)$$

$$= \arg \min_{\mathbf{c}} -2\mathbf{x}^\top \mathbf{D}\mathbf{c} + \mathbf{c}^\top \mathbf{I}_l \mathbf{c} \quad (2.61)$$

(в силу ограничений на ортогональность и нормированность $\mathbf{D}$)

$$= \arg \min_{\mathbf{c}} -2\mathbf{x}^\top \mathbf{D}\mathbf{c} + \mathbf{c}^\top \mathbf{c}. \quad (2.62)$$

Для решения этой задачи оптимизации мы можем воспользоваться векторным математическим анализом (если вы не знаете, что это такое, обратитесь к разделу 4.3):

$$\nabla_{\mathbf{c}}(-2\mathbf{x}^\top \mathbf{D}\mathbf{c} + \mathbf{c}^\top \mathbf{c}) = 0 \quad (2.63)$$

$$-2\mathbf{D}^\top \mathbf{x} + 2\mathbf{c} = 0 \quad (2.64)$$

$$\mathbf{c} = \mathbf{D}^\top \mathbf{x}. \quad (2.65)$$

Получается очень эффективный алгоритм: для оптимального кодирования $\mathbf{x}$ достаточно всего лишь операций над матрицами и векторами, т. е. функция кодирования выглядит так:

$$f(\mathbf{x}) = \mathbf{D}^\top \mathbf{x}. \quad (2.66)$$

Еще раз применив умножение на матрицу, мы сможем определить операцию реконструкции в алгоритме PCA:

$$r(\mathbf{x}) = g(f(\mathbf{x})) = \mathbf{D}\mathbf{D}^\top \mathbf{x}. \quad (2.67)$$

Далее следует выбрать кодировочную матрицу $\mathbf{D}$. Для этого вернемся к идее минимизации расстояния в смысле нормы L^2 между исходными и реконструированными точками. Поскольку для декодирования всех точек используется одна и та же матрица $\mathbf{D}$, мы больше не можем рассматривать точки изолированно, а должны минимизировать норму Фробениуса матрицы ошибок, вычисленных для всех измерений и точек:

$$\mathbf{D}^* = \arg \min_{\mathbf{D}} \sqrt{\sum_{i,j} (x_j^{(i)} - r(\mathbf{x}^{(i)})_j)^2} \text{ при условии } \mathbf{D}^\top \mathbf{D} = \mathbf{I}_l. \quad (2.68)$$

Для вывода алгоритма нахождения $\mathbf{D}^*$ сначала рассмотрим случай $l = 1$. Тогда $\mathbf{D}$ — это просто одиночный вектор, который мы обозначим $\mathbf{d}$. Подставляя (2.67) в (2.68) и заменяя $\mathbf{D}$ на $\mathbf{d}$, мы сводим задачу к такой:

$$\mathbf{d}^* = \arg \min_{\mathbf{d}} \sum_i \|\mathbf{x}^{(i)} - \mathbf{d}\mathbf{d}^\top \mathbf{x}^{(i)}\|_2^2 \text{ при условии } \|\mathbf{d}\|_2 = 1. \quad (2.69)$$

Этот прямой результат подстановки — не самый красивый способ записи уравнения. Скалярное значение $\mathbf{d}^\top \mathbf{x}^{(i)}$ находится справа от вектора $\mathbf{d}$. Но обычно скалярные коэффициенты записываются слева от вектора, поэтому перепишем выражение в таком виде:

$$\mathbf{d}^* = \arg \min_{\mathbf{d}} \sum_i \|\mathbf{x}^{(i)} - \mathbf{d}^\top \mathbf{x}^{(i)} \mathbf{d}\|_2^2 \text{ при условии } \|\mathbf{d}\|_2 = 1. \quad (2.70)$$

или, воспользовавшись тем, что операция транспонирования не изменяет скаляр:

$$\mathbf{d}^* = \arg \min_{\mathbf{d}} \sum_i \|\mathbf{x}^{(i)} - \mathbf{x}^{(i)\top} \mathbf{d} \mathbf{d}\|_2^2 \text{ при условии } \|\mathbf{d}\|_2 = 1. \quad (2.71)$$

Читателю следует стремиться к свободному владению такими косметическими преобразованиями.

Теперь было бы полезно переформулировать задачу в терминах одной матрицы примеров, а не суммы по отдельным векторам. Это позволит записать ее более компактно. Обозначим $\mathbf{X} \in \mathbb{R}^{m \times n}$ матрицу, образованную всеми векторами, рассматриваемыми как строки, т. е. $\mathbf{X}_{i,:} = \mathbf{x}^{(i)\top}$. Тогда уравнение (2.71) можно переписать в виде:

$$\mathbf{d}^* = \arg \min_{\mathbf{d}} \|\mathbf{X} - \mathbf{X} \mathbf{d} \mathbf{d}^\top\|_F^2 \text{ при условии } \mathbf{d}^\top \mathbf{d} = 1. \quad (2.72)$$

Забудем ненадолго об ограничении и упростим часть, содержащую норму Фробениуса:

$$\arg \min_{\mathbf{d}} \|\mathbf{X} - \mathbf{X} \mathbf{d} \mathbf{d}^\top\|_F^2 \quad (2.73)$$

$$= \arg \min_{\mathbf{d}} \text{Tr}((\mathbf{X} - \mathbf{X} \mathbf{d} \mathbf{d}^\top)^\top (\mathbf{X} - \mathbf{X} \mathbf{d} \mathbf{d}^\top)) \quad (2.74)$$

(в силу формулы (2.49))

$$= \arg \min_{\mathbf{d}} \text{Tr}(\mathbf{X}^\top \mathbf{X} - \mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top - \mathbf{d} \mathbf{d}^\top \mathbf{X}^\top \mathbf{X} + \mathbf{d} \mathbf{d}^\top \mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) \quad (2.75)$$

$$= \arg \min_{\mathbf{d}} \text{Tr}(\mathbf{X}^\top \mathbf{X}) - \text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) - \text{Tr}(\mathbf{d} \mathbf{d}^\top \mathbf{X}^\top \mathbf{X}) + \text{Tr}(\mathbf{d} \mathbf{d}^\top \mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) \quad (2.76)$$

$$= \arg \min_{\mathbf{d}} - \text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) - \text{Tr}(\mathbf{d} \mathbf{d}^\top \mathbf{X}^\top \mathbf{X}) + \text{Tr}(\mathbf{d} \mathbf{d}^\top \mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) \quad (2.77)$$

(поскольку члены, не содержащие $\mathbf{d}$, на влияют на $\arg \min$)

$$= \arg \min_{\mathbf{d}} - 2\text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) + \text{Tr}(\mathbf{d} \mathbf{d}^\top \mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) \quad (2.78)$$

(поскольку матрицы внутри оператора следа можно циклически переставлять, формула (2.52))

$$= \arg \min_{\mathbf{d}} - 2\text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) + \text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top \mathbf{d} \mathbf{d}^\top) \quad (2.79)$$

(еще раз применяя то же свойство).

Теперь снова вспомним об ограничении:

$$\arg \min_{\mathbf{d}} - 2\text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) + \text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top \mathbf{d} \mathbf{d}^\top) \text{ при условии } \mathbf{d}^\top \mathbf{d} = 1 \quad (2.80)$$

$$= \arg \min_{\mathbf{d}} - 2\text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) + \text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) \text{ при условии } \mathbf{d}^\top \mathbf{d} = 1 \quad (2.81)$$

(в силу ограничения)

$$= \arg \min_{\mathbf{d}} - \text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) \text{ при условии } \mathbf{d}^\top \mathbf{d} = 1 \quad (2.82)$$

$$= \arg \max_{\mathbf{d}} \text{Tr}(\mathbf{X}^\top \mathbf{X} \mathbf{d} \mathbf{d}^\top) \text{ при условии } \mathbf{d}^\top \mathbf{d} = 1 \quad (2.83)$$

$$= \arg \max_{\mathbf{d}} \text{Tr}(\mathbf{d}^\top \mathbf{X}^\top \mathbf{X} \mathbf{d}) \text{ при условии } \mathbf{d}^\top \mathbf{d} = 1 \quad (2.84)$$